

Aula 02

*DataPrev (Perfil 3: Desenvolvimento de
Software) Banco de Dados - 2026
(Pós-Edital)*

Autor:

Lucas Fargoni Di Ianni

09 de Julho de 2026

Índice

1) 3.0 Normalização - Teoria	3
2) 3.0 Normalização - Resumo	33
3) 3.0 Normalização - Questões Comentadas - FGV	40
4) 3.0 Normalização - Lista de Questões - FGV	66

NORMALIZAÇÃO

Introdução

O processo de normalização de banco de dados relacionais foi proposto pelo próprio criador do modelo relacional, Edgar F. Codd, quando ainda trabalhava na IBM, para garantir que os bancos construídos sob seu modelo fossem bem estruturados.

Assim sendo, com a **normalização** conseguimos **organizar como os dados de um banco relacional serão armazenados**, permitindo que as restrições de integridade do banco (como chaves primárias, chaves estrangeiras, regras de unicidade etc.) consigam os proteger de fato.

Isso porque, na prática, mesmo seguindo as restrições impostas pelo modelo relacional, as colunas das tabelas de um banco relacional podem enfrentar problemas de dependências indesejáveis, isto é, há situações onde uma informação afeta outra de forma imprevisível e problemática.

Na literatura, definem-se três tipos dessas situações indesejadas que podem ocorrer, chamadas de anomalias. Vejamos.

Tipo de anomalia	Causa	Consequência
Inserção	A inserção de dado novo depende de outro que não existe ainda.	Impossibilidade de registrar informações novas.

A anomalia de inserção acontece quando a estrutura mal projetada de uma tabela cria uma situação onde é impossível registrar uma informação válida, simplesmente porque outros dados, que não existem ainda ou não são relevantes naquele momento, são obrigatórios para que o registro exista.

Por exemplo, imaginem uma tabela "Profissionais e Dependentes" que mistura dados dos funcionários de uma empresa com dados de seus familiares, com os valores de CPF de ambos compondo uma chave primária.

Funcionários e Dependentes				
<u>CPF funcionário</u>	nome funcionário	<u>CPF dependente</u>	nome dependente	parentesco
111	Maria	999	Mario	cônjuge
111	Maria	888	João	filho
333	Joana	777	José	cônjuge

Tipo de anomalia	Causa	Consequência
------------------	-------	--------------

Agora imaginem que um novo funcionário foi contratado, porém sem nenhum dependente. Será simplesmente impossível o inserir na nossa tabela, porque ela exige a existência de um CPF de dependente (já que é um atributo chave primária).

Isso ocorre porque "Funcionário" e "Dependente" são entidades distintas que forçamos coexistirem na mesma tabela, criando, assim, uma dependência artificial entre suas informações que não existe na realidade.

Funcionários e Dependentes				
CPF <u>funcionário</u>	nome funcionário	CPF <u>dependente</u>	nome dependente	parentesco
111	Maria	999	Mario	cônjuge
111	Maria	888	João	filho
333	Joana	777	José	cônjuge
444	Ana	Null	Null	Null

Impossível de adicionar um novo funcionário que não tenha dependentes, pois o campo "CPF dependente" é obrigatório (chave primária)

Exclusão	Há atributos de entidades diferentes misturados na mesma tabela.	Ao apagarmos um registro, perdemos acidentalmente informações de outras entidades.
-----------------	--	--

A anomalia de exclusão acontece quando, ao remover um registro que deveria ser apagado, o banco elimina junto informações de outra entidade que deveriam continuar existindo.

É um efeito colateral destrutivo causado, assim como a anomalia de inserção, pela mistura indevida de entidades distintas em uma única tabela.

Ainda no nosso exemplo da tabela com "Funcionários" e seus "Dependentes" juntos, ao retirarmos uma relação de dependência que não existe mais, como o cônjuge de um funcionário (em caso de divórcio), apagamos junto os dados daquele funcionário também.

Tipo de anomalia	Causa	Consequência
------------------	-------	--------------

Funcionários e Dependentes				
CPF funcionário	nome funcionário	CPF dependente	nome dependente	parentesco
111	Maria	999	Mario	cônjuge
111	Maria	888	João	filho
333	Joana	777	José	cônjuge

Para apagarmos as informações de José, precisamos apagar também as de Joana, visto que ambas coexistem no mesmo registro

Atualização	Há o mesmo dado repetido em várias linhas da tabela.	Ao atualizarmos um registro, podemos gerar inconsistências e dados contraditórios.
--------------------	--	--

A anomalia de atualização acontece quando um mesmo dado está repetido em várias linhas da tabela e, ao tentar atualizá-lo, a alteração é aplicada em apenas algumas dessas linhas, deixando o banco com versões diferentes e contraditórias da mesma informação.

No exemplo da tabela com "Funcionários" e seus "Dependentes" juntos, podemos ter um funcionário com dois dependentes, um cônjuge e um filho e, ao tentarmos atualizar o telefone dele, por qualquer problema (erro humano, queda de conexão, bug no sistema etc.), atualiza-se apenas uma das linhas.

Tipo de anomalia	Causa	Consequência
------------------	-------	--------------

Funcionários e Dependentes				
CPF funcionário	nome funcionário	CPF dependente	nome dependente	parentesco
111	Maria B.	999	Mario	cônjuge
111	Maria	888	João	filho
333	Joana	777	José	cônjuge

Por algum problema, apenas o primeiro registro de Maria foi atualizado para Maria B.

Para resolvermos esses problemas, aplicamos justamente o processo de normalização, que aplica, sucessivamente, um conjunto de regras chamadas Formas Normais.

Nesse processo, cada etapa, isto é, cada Forma Normal, aplica critérios cada vez mais rigorosos a serem cumpridos pelas tabelas do banco, para que, ao final do processo, os dados estejam bem organizados.



É importante sabermos desde já que **cada forma normal herda todas as regras das anteriores e adiciona novas exigências.**

Por exemplo, para uma tabela estar na 2ª Forma Normal, é necessário que cumpra com as regras da 1ª Forma normal e também com suas próprias regras.

Ou seja, toda tabela que estiver na 2ª Forma Normal, estará também na 1ª Forma Normal. Dessa mesma forma, toda tabela na 3ª Forma Normal, está na 2ª e 1ª Forma Normal.

Além disso, ao aplicarmos as Formas Normais, obtemos um banco de dados com muitos benefícios, vejamos.

Benefício	Explicação
Prevenção de Anomalias de Dados	Com a normalização, a alteração de um banco de dados torna-se um processo mais confiável, sendo menos propenso a erros.
Redução da Redundância Não Intencional	Dados duplicados por acidente, fruto de design ruim, geram inconsistências e desperdícios. Com a normalização, combatemos esse tipo de problema.
Economia nos Custos de Armazenamento	Cada dado duplicado ocupa espaço físico real, seja em disco local ou em servidores na nuvem. Em ambientes de nuvem como AWS, Google Cloud ou Azure, onde pagamos exatamente pelo que usamos, esse benefício se traduz em economia financeira direta.
Recuperação Mais Rápida de Dados	Quando um banco de dados tem menos dados redundantes, as consultas se tornam mais eficientes, exigindo menor poder de processamento.



(CEBRASPE (CESPE)/APO (MPO)/MPO/Tecnologia da Informação/Gestão de Dados Orçamentários/2024) Normalização de dados é um processo que organiza os dados em um banco de dados para minimizar a redundância e promover a integridade.

Comentários:

O processo de normalização serve justamente para organizar como os dados de um banco relacional serão armazenados, aplicando sucessivamente um conjunto de regras chamadas Formas Normais. Ao fazer isso, conseguimos reduzir redundâncias não intencionais, ou seja, dados duplicados por acidente que geram inconsistências e desperdícios. Além disso, a normalização permite que as restrições de integridade do banco, como chaves primárias, chaves estrangeiras e regras de unicidade, consigam proteger os dados de fato, prevenindo as chamadas anomalias de inserção, exclusão e atualização.

Gabarito: Certo

(CEBRASPE (CESPE)/Ana Proc (DATAPREV)/DATAPREV/2023) Um banco de dados que se encontra na terceira forma normal obrigatoriamente está também na segunda e na primeira formas normais.

Comentários:

Isso está correto porque o processo de normalização funciona de forma acumulativa, ou seja, cada forma normal herda todas as regras das anteriores e adiciona novas exigências. Para uma tabela estar na segunda forma normal, ela precisa primeiro cumprir as regras da primeira forma normal. Da mesma maneira, para estar na terceira forma normal, ela precisa antes cumprir as regras da segunda forma normal, que por sua vez já exige o cumprimento da primeira. Então, toda tabela que está na terceira forma normal automaticamente está também na segunda e na primeira formas normais.

Gabarito: Certo

Dependências Funcionais

Definimos algumas Formas Normais (a 2ª, a 3ª e a de Boyce Codd) a partir do conceito de "Dependências Funcionais", portanto passemos a estudá-las antes de entrarmos no processo de normalização em si.

Uma **Dependência Funcional** (DF) existe quando os **valores de algumas colunas definem obrigatoriamente os valores de outras**. Ou seja, com uma dependência funcional conseguimos saber exatamente quais atributos são necessários para determinarmos outros.

Por exemplo, se estabelecermos a dependência funcional "CPF determina nome", saberemos que, ao possuímos um "CPF" de uma pessoa, necessariamente saberemos qual o "nome" dela.



Notem, então, algo importante: definimos as dependências funcionais a partir das regras do negócio. Assim, dependências funcionais são verdades do mundo real.

Em questões de concursos, ou a dependência funcional é tão óbvia que não precisa ser dita (como "CPF" determina "nome"), ou ela é afirmada expressamente (como, na administração tributária, "Período" e "Tipo de contribuinte" definem o "regime tributário" que ele está sujeito.)

Por isso também que, muitas vezes, não conseguimos olhar para uma tabela e saber se ela respeita determinada Forma Normal. É necessário que as dependências funcionais sejam fornecidas para que isso seja possível de ser analisado.

Assim sendo, com as dependências funcionais temos as garantias estruturais estabelecidas pelo mundo real que o banco deve respeitar em todos os registros, presentes e futuros.

Por exemplo, para "CPF determina nome", isso significa que, no banco, jamais poderão existir dois registros com o mesmo "CPF" e "nome" diferentes. O banco tem de tratar nossas regras de negócio como uma lei.

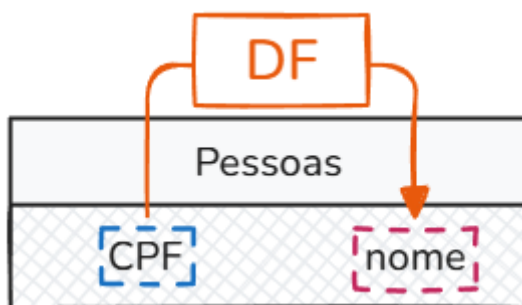
Mundo real → Regras do negócio → Dependências funcionais → Estrutura do banco

Continuando, a **notação** usada para representar dependências funcionais é "**A → B**" ("A determina B" ou "B depende de A"). Para o nosso exemplo, teríamos "CPF → nome".

Além disso, chamamos os atributos do lado esquerdo da dependência funcional de "Conjunto Determinante" (a causa na relação) e do lado direito de "Conjunto Dependente".



Finalmente, eu gosto de representar a dependência funcional graficamente a partir de setas entre os atributos de uma tabela, vejam.



(FGV/Ana Esp (IMBEL)/IMBEL/Analista de Sistemas/2021) Na teoria de projetos de bancos de dados, o processo de normalização de uma tabela é feito a partir da identificação

- a) das chaves estrangeiras.
- b) das dependências funcionais.
- c) das junções que serão necessárias.
- d) dos tipos de dados em cada coluna.
- e) dos relacionamentos entre as tabelas.

Comentários:

- a) **Errada.** Chaves estrangeiras são restrições de integridade do banco, mas não são o elemento central usado para identificar como normalizar uma tabela.
- b) **Correta.** Várias formas normais (a 2FN, a 3FN e a Forma Normal de Boyce Codd) são definidas justamente a partir do conceito de dependências funcionais. São elas que nos permitem analisar quais atributos determinam outros e, assim, identificar problemas como dependências parciais e transitivas que precisam ser corrigidos no processo de normalização.
- c) **Errada.** Junções são operações usadas para consultar dados, não são a base para identificar como normalizar uma tabela.
- d) **Errada.** Os tipos de dados das colunas (texto, número, data etc.) dizem respeito ao armazenamento dos valores, mas não são o fundamento para o processo de normalização.
- e) **Errada.** Relacionamentos entre tabelas são importantes no modelo relacional, mas o processo de normalização de uma tabela se baseia na identificação das dependências funcionais entre seus atributos, e não nos relacionamentos entre tabelas diferentes.

Gabarito: B

(VUNESP/DPE SP/Analista Desenvolvedor/2023) A teoria de dependências funcionais é utilizada no projeto de bancos de dados. Desta forma, considere uma relação R e dois subconjuntos arbitrários, X e Y, do conjunto de atributos de R. A notação utilizada para determinar que o subconjunto X determina funcionalmente o subconjunto Y é:

- a) $X \otimes Y$.
- b) $Y \otimes X$.
- c) $X > Y$.
- d) $X \rightarrow Y$.
- e) $Y \rightarrow X$.

Comentários:

Para determinar que um conjunto de atributos X determina outro conjunto de atributos Y utilizamos uma seta entre eles, nessa ordem: $X \rightarrow Y$.

Gabarito: D

Manipulando Dependências Funcionais

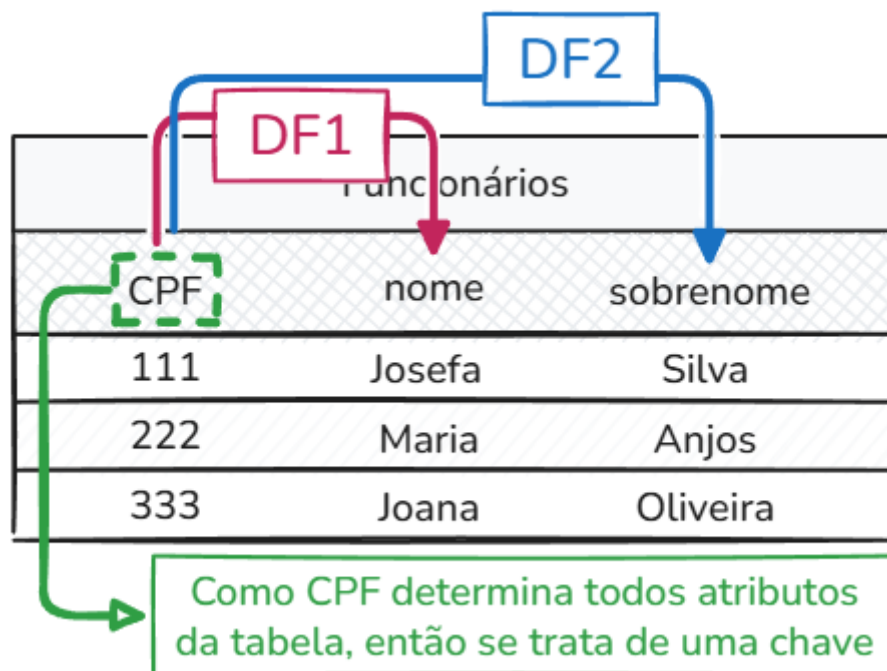
Além de entender o que é uma dependência funcional, precisamos aprender a trabalhar com elas para descobrir novas dependências a partir das que já conhecemos.

Mas por que isso é importante, professor? Porque, no dia a dia, quem projeta o banco de dados costuma receber do cliente apenas as dependências mais evidentes. Só que, para conseguir montar o banco corretamente, precisamos conhecer todas elas, inclusive as que não são tão óbvias.

E entre as várias vantagens de ter essa lista completa, uma se destaca bastante (especialmente em concursos): a possibilidade de identificar as chaves de uma tabela diretamente a partir das dependências funcionais.

Isso porque, se um conjunto de atributos consegue determinar todos outros atributos da tabela, então ele identifica o registro inteiro, ou seja, trata-se de uma chave!

A seguir, temos uma tabela com as dependências $\text{CPF} \rightarrow \text{nome}$ e $\text{CPF} \rightarrow \text{sobrenome}$, vejam.



Pois bem, para manipularmos as dependências funcionais utilizamos algumas regras, chamadas de Axiomas de Armstrong, estudadas a seguir.

Axiomas de Armstrong

Reflexividade

Um conjunto de atributos sempre determina suas partes

Por exemplo, se temos o "CPF" e o "nome" de uma pessoa, então esses atributos com certeza determinam o próprio "CPF", o próprio "nome" e também o "CPF" e o "nome".

$$\begin{array}{lcl}
 & & \{\text{CPF}, \text{nome}\} \rightarrow \text{CPF} \\
 \{\text{CPF}, \text{nome}\} & \Rightarrow & \{\text{CPF}, \text{nome}\} \rightarrow \text{nome} \\
 & & \{\text{CPF}, \text{nome}\} \rightarrow \{\text{CPF}, \text{nome}\}
 \end{array}$$

Se tivermos só um atributo, a reflexividade pode ser escrita como uma dependência funcional trivial (isto é, óbvia): *um atributo sempre determina ele mesmo*.

Por exemplo, se eu sei o CPF de uma pessoa, então eu consigo saber o CPF dela (!!!), ou seja, o "CPF determina CPF".

Eu sei que parece uma regra muito boba, mas ela nos dá uma ferramenta poderosa para encontrarmos as chaves de uma tabela.

Isso porque, como um atributo sempre determina ele mesmo, se ele aparecer do lado esquerdo de uma dependência funcional, podemos o passar para o lado direito sem problemas.

Por exemplo, imaginem uma tabela com 3 atributos, "CPF", "RG" e "nome", e uma Dependência Funcional, "CPF" e "RG" determinam o "nome". Então, só com essa DF e com a reflexividade, sabemos que "CPF" e "RG" formam uma superchave.

Como assim professor?!?!? Olha só, temos que "{CPF, RG} → nome", e sabemos que todo atributo determina ele mesmo, então podemos passar CPF e RG para o lado direito da dependência funcional: "{CPF, RG} → {CPF, RG, nome}".

Agora vejam que os atributos "{CPF, RG}" determinaram todos atributos da tabela "{CPF, RG, nome}", ou seja, trata-se de uma superchave!

Augmentação

Podemos adicionar atributo extra à toda dependência sem a invalidar

Por exemplo, em uma tabela com três atributos, "CPF", "nome" e "sobrenome", se temos que o "CPF" determina o "nome", então posso adicionar um atributo "sobrenome" dos dois lados da dependência funcional, preservando sua validade.

CPF → nome ⇒ {CPF, sobrenome} → {nome, sobrenome}

Na prática, utilizamos essa regra para "completar" conjuntos determinantes até virarem chaves.

No próprio exemplo anterior, usamos a augmentação para transformar "CPF → nome" em "{CPF, sobrenome} → {nome, sobrenome}" e, da regra da reflexividade, podemos passar CPF para o lado direito da dependência funcional, criando a dependência funcional "{CPF, sobrenome} → {CPF, nome, sobrenome}". Portanto, os atributos "CPF" e "sobrenome" determinam todos os outros, se constituindo em uma chave.

Transitividade

Podemos encadear duas dependências se tiverem atributos iguais no determinante de uma e determinado da outra

Por exemplo, em uma tabela com três atributos, "CPF", "CEP" e "endereço", se temos que o "CPF" determina o "CEP" e "CEP" determina o "endereço", então podemos deduzir que "CPF" determina o "endereço".

CPF → CEP ⇒ CPF → endereço
CEP → endereço

Na prática, utilizamos essa propriedade para encontrarmos chaves “escondidas”. Porém, para explicar isso, precisamos de outro exemplo.

Vamos supor que já temos um atributo que determine todos os outros da tabela (ou seja, uma chave), como $\text{CPF} \rightarrow \{\text{CPF}, \text{RG}, \text{nome}, \text{sobrenome}\}$, e também um atributo que determina essa chave, como $\text{RG} \rightarrow \text{CPF}$.

Da transitividade, podemos concluir então que $\text{RG} \rightarrow \{\text{CPF}, \text{RG}, \text{nome}, \text{sobrenome}\}$, ou seja, RG também é chave!



(FGV/MPE SC/Tecnologia da Informação/2022) Na teoria do projeto de bancos de dados relacionais, o estabelecimento das dependências funcionais tem um papel importante, pois é a partir dessas relações que é possível proceder à normalização dos bancos. Essas dependências funcionais podem ser manipuladas por meio de um conjunto básico de axiomas.

Com relação aos atributos X, Y, W e Z, analise as seguintes derivações propostas.

I. Se $X \rightarrow Y$ e $Y \rightarrow Z$ então $X \rightarrow Z$.

II. Se $X \rightarrow Y$ então $XW \rightarrow ZW$.

III. Se Y está contido num conjunto qualquer de atributos, como XYZW, por exemplo, então $XYZW \rightarrow Y$.

Sobre essas derivações, está correto o que é proposto em:

- a) I, somente;
- b) I e II, somente;
- c) I e III, somente;
- d) II e III, somente;
- e) I, II e III.

Comentários:

I. Se $X \rightarrow Y$ e $Y \rightarrow Z$ então $X \rightarrow Z$.

Verdadeira. É a regra da transitividade. Se X determina Y, e Y determina Z, então X também determina Z indiretamente.

II. Se $X \rightarrow Y$ então $XW \rightarrow ZW$.

Falsa. É a regra da augmentação. O lado esquerdo acrescenta W corretamente, mas o lado direito deveria ser YW, não ZW. O correto seria: $XW \rightarrow YW$.

III. Se Y está contido em XYZW, então $XYZW \rightarrow Y$.

Verdadeira. É a regra da reflexividade. Um conjunto de atributos sempre determina funcionalmente qualquer subconjunto seu. Se Y faz parte de XYZW, então $XYZW \rightarrow Y$ é sempre válido.

Gabarito: C

Dos Axiomas de Armstrong podemos derivar mais algumas regras bem úteis para acelerarmos nossa resolução de questões de concursos, vejam.

União

**Se um atributo determina outros separadamente,
então também os determina juntos**

Por exemplo, se temos que o "CPF" determina o "nome" de uma pessoa e também que "CPF" determina o "sobrenome", então o "CPF" determina o "nome" e o "sobrenome" ao mesmo tempo.

$$\begin{array}{l} \text{CPF} \rightarrow \text{nome} \\ \text{CPF} \rightarrow \text{sobrenome} \end{array} \Rightarrow \text{CPF} \rightarrow \{\text{nome}, \text{sobrenome}\}$$

Ou seja, quando um mesmo determinante gera duas dependências, podemos unir os dependentes no lado direito.

Decomposição

**Se um atributo determina outros juntos,
então também os determina separadamente**

Por exemplo, se o "CPF" determina o "nome" e o "sobrenome" de uma pessoa ao mesmo tempo, então podemos afirmar que o "CPF" determina o "nome" e também que o "CPF" determina o "sobrenome".

$$\text{CPF} \rightarrow \{\text{nome}, \text{sobrenome}\} \Rightarrow \begin{array}{l} \text{CPF} \rightarrow \text{nome} \\ \text{CPF} \rightarrow \text{sobrenome} \end{array}$$

Ou seja, quando um determinante gera um conjunto de atributos dependentes, podemos separar os dependentes no lado direito.

Formas Normais

1ª Forma Normal

A 1ª Forma Normal (1FN) constitui a primeira etapa no processo de normalização, sendo o primeiro conjunto de regras a serem cumpridas.

Bom, a **primeira regra da 1FN** exige que **cada tabela** do banco relacional **contenha**, necessariamente, uma **chave primária**. Consequentemente, cada registro de cada tabela deve ser sempre identificável de forma única.

Além disso, essa forma normal **também exige a atomicidade dos atributos** das tabelas, ou seja, exige que cada campo deve conter sempre um único valor indivisível.

Um atributo é atômico quando não pode ser dividido em partes menores dentro do contexto do banco de dados.

A 1FN também proíbe a existência dos chamados **grupos repetitivos** (ou grupos de repetição), isto é, **conjuntos de atributos que aparecem múltiplas vezes dentro de uma mesma tabela ou atributo**, seja como múltiplos valores num campo, como colunas duplicadas, ou como estruturas aninhadas. Vamos estudar cada uma dessas estruturas a seguir.

Primeiramente, sabemos que **não** pode haver **atributos multivalorados**, que ocorrem quando uma única célula contém múltiplos valores do mesmo tipo. Por exemplo, não pode existir um atributo "telefones" que permite a inserção de vários telefones no mesmo campo da tabela.

Pessoas		
CPF	nome	telefones
111	Maria B.	9871-4751, 9471-1274, 9544-4756
222	Maria	9841-5151
333	Joana	9157-7474, 9334-2798

Atributo Multivalorado

Também **não** podem existir **atributos compostos**, isto é, atributos que aceitam múltiplas informações de tipos diferentes que poderiam ser separadas. O exemplo clássico da literatura de atributo composto é o "endereço", que geralmente compõe-se por rua, número, complemento, cidade e estado (informações distintas misturadas nesse um único campo).

Pessoas		
CPF	nome	endereço
111	Maria B.	Rua um, nº 57, apto 3, São Luís - MA
222	Maria	Rua couve, nº 44, São Gonçalo - RJ
333	Joana	Rua pássaros, nº 101, Inocência - MS

Atributo Composto

Além disso, **não** pode haver **atributos repetitivos**, ou seja, a mesma informação aparecendo várias vezes na tabela para representar múltiplos valores. Por exemplo, não podemos ter atributos "telefone1", "telefone2", "telefone3"... para permitir a inserção de vários telefones na mesma tabela.

Pessoas				
CPF	nome	telefone1	telefone2	telefone3
111	Maria B.	9871-4751	9471-1274	9544-4756
222	Maria	9841-5151	Null	Null
333	Joana	9157-7474	9334-2798	Null

Atributos Repetitivos

Finalmente, **não** pode haver **tabelas aninhadas**, ou seja, estruturas hierárquicas onde um campo de uma tabela contém dentro dela uma outra tabela completa (como "tabela dentro de tabela"), formando, na prática, conjuntos de valores associados a um atributo apenas.

Pessoas							
CPF	nome	telefones					
111	Maria B.	<table><tr><th>Telefones</th></tr><tr><th>telefone</th></tr><tr><td>9871-4751</td></tr><tr><td>9841-5151</td></tr><tr><td>9157-7474</td></tr></table>	Telefones	telefone	9871-4751	9841-5151	9157-7474
Telefones							
telefone							
9871-4751							
9841-5151							
9157-7474							

Tabela Aninhada

Tabela Aninhada

Essa última regra pode parecer meio esquisita, mas surgiu para combater justamente o que **os bancos de dados hierárquicos** (predominantes antes dos relacionais surgirem) permitiam: árvores de dados em que registros “pai” continham registros “filho” diretamente dentro deles (como se fossem tabelas dentro de tabelas).



Atualmente esse tipo de estrutura de dados é típico de bancos NoSQL orientados a documentos, como o MongoDB, onde o aninhamento de estruturas de dados, formando grupos repetitivos (no caso, documentos), não só é permitido, como até esperado.

Agora precisamos avançar em um detalhe histórico e conceitual muito relevante.

Em 1970, a primeira forma normal foi introduzida por Edgar Codd (o criador do modelo relacional) somente como “Forma Normal”, no próprio artigo em que propôs o modelo relacional. Assim sendo, a “Forma Normal” apresentava a definição mínima obrigatória que todo banco relacional precisava obedecer.

Foi só em 1971, ao propor as formas adicionais, que ele precisou renomear a “Forma Normal” original para “1ª Forma Normal”, criando assim a hierarquia 1FN, 2FN, 3FN que conhecemos hoje.

Ocorre que **a ideia que a 1FN (antiga “Forma Normal”) precisa necessariamente ser obedecida para um banco de dados ser considerado relacional não mudou**. Isso porque, se não estiver na 1FN, o modelo relacional perde sua base matemática juntamente com suas garantias fundamentais, e, conseqüentemente, o banco de dados deixa de ser relacional.

Conseqüentemente, **suas regras** não são uma escolha de design do projetista do banco de dados, como as outras formas normais: **precisam necessariamente serem obedecidas**.



(CEBRASPE (CESPE)/Ana Tec (SUSEP)/SUSEP/Tecnologia da Informação e Ciência de Dados/2025) A primeira forma normal (1FN) será satisfeita quando todos os atributos tiverem domínio atômico, ou seja, quando não houver valores compostos ou repetitivos.

Comentários:

A afirmação está correta porque a 1ª Forma Normal exige justamente a atomicidade dos atributos das tabelas, ou seja, cada campo deve conter sempre um único valor indivisível. Isso significa que não podem existir atributos compostos (como um campo "endereço" que mistura rua, número, cidade e estado em uma única célula), nem atributos multivalorados (como um campo "telefones" com vários telefones no mesmo campo), nem atributos repetitivos (como "telefone1", "telefone2", "telefone3" representando a mesma informação em várias colunas). Quando todos os atributos respeitam essa regra de domínio atômico, sem valores compostos ou repetitivos, a tabela satisfaz a 1FN nesse aspecto.

Gabarito: Certo

2ª Forma Normal

Sendo direto ao ponto, para uma tabela estar na **2ª Forma Normal**, além de estar na **1ª Forma Normal**, é necessário que todos **seus atributos que não são chaves devem depender completamente de todos atributos que compõem as chaves**.

Em outras palavras, a **2ª Forma Normal não tolera dependências parciais**, ou seja, **atributos "comuns" que dependem apenas parcialmente dos atributos de uma chave**. Em outras palavras, exige apenas dependências funcionais totais da chave.



Percebam que a 2FN se aplica apenas em tabelas com chave composta, isso porque, se houver apenas chaves com um atributo é impossível haver uma dependência de apenas parte da chave.

Para exemplificar, vamos supor uma tabela de funcionários de uma empresa com seus dependentes, com atributos "CPF_funcionario", "CPF_dependente", "nome_funcionario" e "nome_dependente", sendo a chave primária composta "CPF_funcionario" e "CPF_dependente" (demarquei ela com linha pontilhada em verde).

Funcionários e Dependentes			
Chave Composta	CPF	CPF	nome
	funcionário	dependente	funcionário dependente
	111	999	Josefa Mario
	222	888	Maria João
	333	777	Joana José

Agora percebam algo muito curioso: essa tabela, nesse momento, não viola a 2FN! Isso porque, para avaliarmos, precisamos definir as Dependências Funcionais (DFs) que regem essa relação.

Pois bem, então vamos apresentar as 2 DFs aplicáveis aqui: "CPF_funcionario" determina o "nome_funcionario" e "CPF_dependente" determina "nome_dependente". Vejamos essas Dependências Funcionais na tabela.

<u>CPF funcionário</u>	<u>CPF dependente</u>	nome funcionário	nome dependente
111	999	Josefa	Mario
222	888	Maria	João
333	777	Joana	José

Notem que aqui temos aquilo que a 2FN proíbe, a dependência parcial. Isso porque, existem atributos não chave ("nome_funcionario" e "nome_dependente") que dependem parcialmente da chave: enquanto "nome_funcionario" depende apenas do "CPF_funcionario", "nome_dependente" depende apenas do "CPF_dependente".

Para corrigirmos isso, precisamos decompor essa tabela em tabelas menores, cada uma organizada em torno de uma dependência funcional.

Dessa forma, podemos criar uma tabela apenas com "CPF_funcionario" e "nome_funcionario", para satisfazermos a Dependência Funcional entre esses dois atributos, e outra com "CPF_dependente", "nome_dependente" e "CPF_Funcionário" (chave estrangeira para a tabela Funcionário), para satisfazermos a outra Dependência Funcional.

Funcionários	
CPF <u>funcionário</u>	nome funcionário
111	Josefa
222	Maria
333	Joana

Dependentes		
CPF <u>dependente</u>	nome dependente	CPF funcionário*
999	Mario	111
888	João	222
777	José	333



(CEBRASPE (CESPE)/ERM (ANM)/ANM/Tecnologia da Informação/Ciência de Dados/2025) Quando todas as colunas não chave de uma tabela dependem de toda a chave composta, essa tabela está na segunda forma normal (2FN).

Comentários:

A afirmação está correta porque a segunda forma normal exige exatamente isso: que todos os atributos que não são chave dependam completamente de todos os atributos que compõem a chave, ou seja, não pode haver dependência parcial. Dependência parcial acontece quando um atributo não chave depende de apenas uma parte da chave composta, e não dela inteira.

Gabarito: Certo

(FGV/AMAZUL/Técnico de Informática/2026) No processo de projeto lógico de bancos de dados relacionais, a normalização organiza os atributos das relações de forma a eliminar redundâncias indesejadas e anomalias de atualização.

Dentro desse encadeamento, a forma normal que tem sua definição fundamentada direta e exclusivamente no conceito de dependência funcional total é a

- a) primeira forma normal (1FN).
- b) segunda forma normal (2FN).
- c) terceira forma normal (3FN).
- d) forma normal Boyce-Codd (FNBC).
- e) quarta forma normal (4FN).

Comentários:

a) Errada. A 1FN se fundamenta em exigências como chave primária, atomicidade dos atributos e ausência de grupos repetitivos, não no conceito de dependência funcional total.

- b) **Correta.** A 2FN exige que todos os atributos que não são chave dependam completamente de todos os atributos que compõem a chave, ou seja, proíbe dependências parciais. Isso é exatamente o conceito de dependência funcional total.
- c) **Errada.** A 3FN se fundamenta no conceito de dependência transitiva, proibindo que atributos comuns determinem outros atributos comuns, e não no conceito de dependência funcional total.
- d) **Errada.** A FNBC se fundamenta na ideia de que apenas chaves podem determinar outros atributos, fechando uma brecha da 3FN. Seu foco não é a dependência funcional total.
- e) **Errada.** A 4FN se fundamenta no conceito de dependência multivalorada, que trata de um valor determinando um conjunto inteiro de outros valores, algo diferente de dependência funcional total.

Gabarito: B

3ª Forma Normal

Originalmente, Codd definiu que, para uma tabela na 3ª Forma Normal (3FN), além de estar na 2FN, também não pode haver atributos "comuns" (não chave) determinando outros atributos "comuns".

Consequentemente, uma tabela na 3FN não possui atributos que dependam indiretamente da chave. Ou seja, não é possível ter uma estrutura de dependência do tipo "atributo chave" determina "atributo comum" que, por sua vez, determina outro "atributo comum".

Essa cadeia de dependências "chave → comum → comum" é conhecida pelo nome técnico dependência transitiva. Assim sendo, a 3FN abomina dependências transitivas.

Há Dependência Transitiva quando uma informação depende de outra informação, que por sua vez depende da chave.

Por exemplo, imaginem uma tabela de funcionários com atributos "id funcionário", "nome", "id departamento" e "nome departamento", com chave primária "id funcionário" e Dependências Funcionais "id funcionário → nome", "id funcionário → id departamento", "id departamento → nome departamento".

Percebam então que temos uma dependência transitiva aqui: "id funcionário" determina "id departamento" e esse último determina "nome departamento". Ou seja, não estamos cumprindo com a 3FN.

Funcionários e Departamentos			
<u>id funcionário</u>	nome	id departamento	nome departamento
111	Josefa	1	Exatas
222	Maria	2	Biológicas
333	Joana	3	Humanas

Dependência Transitiva

Para corrigir isso, basta separarmos a tabela original em tabelas novas com base nas dependências que geram a transitividade. Vamos, então, manter em uma tabela as DFs "id funcionário → nome" e "id funcionário → id departamento" e, em outra, vamos criar a "id departamento → nome departamento".

Funcionários		
<u>id funcionário</u>	nome	id departamento*
111	Josefa	1
222	Maria	2
333	Joana	3

Departamento	
<u>id departamento</u>	nome departamento
1	Exatas
2	Biológicas
3	Humanas



(CEBRASPE (CESPE)/AJ TRF6/TRF 6/Apoio Especializado/Governança e Gestão de Tecnologia da Informação/2025) Uma tabela está na 3.ª forma normal (3NF) se todos os atributos dependerem diretamente da chave primária.

Comentários:

Para uma tabela estar na 3ª Forma Normal, ela precisa, antes de tudo, estar na 2ª Forma Normal. Portanto, não basta que todos atributos dependam diretamente da chave primária.

Gabarito: Errado

Forma Normal de Boyce Codd

A Forma Normal de Boyce Codd (FNBC) é frequentemente chamada de 3.5FN, um nome informal que resume bem sua posição: ela vai além da 3FN, mas não chega a ser um nível completamente novo. É mais um refinamento que fecha uma lacuna específica deixada pela 3FN.

Então, para entendermos por que a FNBC foi criada, precisamos antes entender onde a 3FN falha.

Sabemos que a **3FN proíbe** que atributos “comuns” (não chave) dependam de outros atributos “comuns”. Ou seja, *ESPERA-SE* que sempre uma chave determine um atributo.

Porém, analisando mais a fundo, podemos perceber que essa definição deixa uma brecha muito sutil: **podemos ter um atributo “comum” determinando um atributo “chave”!**

Isso porque a 3FN só protege atributos “comuns” de serem determinados por outros atributos “comuns”, permitindo que atributos de chaves sejam determinados por atributos “comuns” sem problemas.

É justamente aí que a Forma Normal de Boyce Codd restringe a 3FN: enquanto na 3FN apenas atributos “comuns” têm de ser determinados por chaves, **na FNBC todos atributos devem ser determinados por chaves.**



Tecnicamente a FNBC permite que o determinante seja apenas uma “superchave”, porém deixaremos a nomenclatura de “chave” nessa explicação para facilitar o entendimento e também porque as bancas não se importam muito com essas nuances técnicas na formulação das questões.

Em outras palavras, na Forma Normal de Boyce Codd apenas chaves podem determinar outro atributo.

Caso	3ª FN	FNBC
comum → comum	✗	✗
comum → chave	✓	✗
chave → comum	✓	✓



(CEBRASPE (CESPE)/TJ TRF6/TRF 6/Apoio Especializado/Desenvolvimento de Sistemas de Informação/2025) Para que uma tabela esteja em BCNF, todos os atributos devem depender unicamente da chave primária, ignorando o impacto de dependências funcionais provenientes de outras superchaves ou chaves candidatas.

Comentários:

A afirmação está errada porque a Forma Normal de Boyce Codd não se limita apenas à chave primária. O que a BCNF exige é que apenas chaves possam determinar outro atributo, ou seja, todo determinante de uma dependência funcional deve ser uma chave. A questão erra ao dizer que se deve "ignorar o impacto de dependências funcionais provenientes de outras superchaves ou chaves candidatas", pois na verdade a BCNF leva em conta todas as chaves, e não somente a chave primária. Se um atributo comum (que não faz parte de nenhuma chave) determinar qualquer outro atributo, inclusive um atributo de chave, a tabela viola a BCNF. Portanto, não se pode ignorar as demais chaves candidatas nessa análise.

Gabarito: Errado

4ª Forma Normal

Até a Forma Normal de Boyce Codd, todas as formas normais combatiam problemas causados por Dependências Funcionais, onde um atributo que determina outro viola alguma restrição.

Na 4FN inauguramos uma nova forma de normalizar nossos dados, observando um tipo de dependência mais complexo: a **dependência multivalorada**.

Mas o que é uma Dependência Multivalorada, professor? Bom, na dependência multivalorada tratamos de **dependências entre um valor determinando um conjunto inteiro de outros valores**.

Como não é muito fácil explicar muito além disso sobre esse conceito, vamos esclarecer com uma visualização. Imaginem, então, uma tabela com os atributos "nome", "matéria" (que leciona) e "linguagem de programação" (que conhece) de uma tabela de professores, sendo nome o determinante de matéria e linguagem de programação.

Professores		
nome	matéria	linguagem de programação
Maria	Matemática	Python
Maria	Matemática	C#
Joana	Português	Erlang
Joana	Português	Javascript
Joana	Literatura	Erlang
Joana	Literatura	Javascript

Agora percebam algo importante: “matéria” e “linguagem de programação” são informações completamente independentes entre si, o que causa em uma multiplicação de linhas com várias informações redundantes, já que todas as combinações possíveis dos valores precisam ser registradas.

Pior ainda acontece quando uma nova linguagem de programação é aprendida: todas matérias precisarão ser replicadas novamente. Por exemplo, vamos supor que Joana aprendeu “Java”.

Professores		
nome	matéria	linguagem de programação
Maria	Matemática	Python
Maria	Matemática	C#
Joana	Português	Erlang
Joana	Literatura	Erlang
Joana	Português	Javascript
Joana	Literatura	Javascript
Joana	Português	Java
Joana	Literatura	Java

Precisamos de 2 registros para adicionar 1 informação: “Joana aprendeu Java”

A solução para esse tipo de problema é separar cada dependência multivalorada em sua própria tabela, estando o determinante (“nome”) presente em ambas.

Professor_Matéria	
nome	matéria
Maria	Matemática
Joana	Português
Joana	Literatura

Professor_Linguagem	
nome	linguagem de programação
Maria	Python
Maria	C#
Joana	Erlang
Joana	Javascript

Vejam que, nessa configuração, adicionar uma nova “linguagem de programação” requer apenas uma linha, e não mais uma linha por “matéria”.



(FGV/Res (TJ RJ)/TJ RJ/Tecnologia da Informação/2024) Uma tabela encontra-se em 4FN quando, além de estar em 3FN, não contém

- a) tabelas aninhadas.
- b) dependências transitivas
- c) tabelas autorelacionadas.
- d) dependências intransitivas.
- e) dependências multivaloradas.

Comentários:

- a) **Errada.** Tabelas aninhadas são proibidas já na 1a Forma Normal, que exige atomicidade e proíbe "tabelas dentro de tabelas".
- b) **Errada.** Dependências transitivas são o problema combatido pela 3a Forma Normal, não pela 4a.
- c) **Errada.** Tabelas autorelacionadas não têm relação com o que a 4a Forma Normal combate.
- d) **Errada.** Dependências intransitivas não são o conceito tratado pela 4FN.
- e) **Correta.** A 4FN exige que, além de estar na 3FN, a tabela não contenha dependências multivaloradas. Dependência multivalorada ocorre quando um valor determina um conjunto inteiro de outros valores, causando uma multiplicação de linhas com informações redundantes. A solução é separar cada dependência multivalorada em sua própria tabela.

Gabarito: E

5ª Forma Normal

A 5FN encerra o ciclo de normalização voltado à eliminação de redundâncias, atacando o tipo de dependência mais sutil de todos: a **dependência de junção problemática**.

Só conseguimos analisar esse tipo de dependência quando há **possibilidade de decompor uma tabela em partes menores e reconstruí-la perfeitamente depois**. Isso porque, uma dependência de junção só existe em uma tabela quando ela puder ser reconstruída (isto é, possuir as mesmas linhas) pela junção de suas tabelas projetadas.

Assim, existir uma dependência de junção em uma tabela não é algo negativo nem positivo, já que só indica que a tabela pode ser reconstruída perfeitamente juntando suas projeções.

Ocorre que, uma tabela está na 5FN se, e somente se, toda dependência de junção nessa tabela for implicada pelas suas chaves candidatas.

Ou seja, o problema surge quando uma dependência de junção não é consequência das chaves da tabela. Nesse caso, temos uma redundância escondida entre os atributos.

Nessa situação, para atingir a 5FN, a tabela deve ser decomposta nas projeções que expressam essa dependência de junção, de modo que as dependências restantes sejam implicadas pelas chaves candidatas.



(CEBRASPE (CESPE)/APO (MPO)/MPO/Tecnologia da Informação/Gestão de Infraestrutura de TI/2024) De acordo com a definição da quinta forma normal (5FN), uma tabela de quarta forma normal (4FN) estará em 5FN quando o conteúdo dela puder ser reconstruído (isto é, quando não existir perda de informação) a partir das diversas tabelas menores que não possuam a mesma chave primária.

Comentários:

Para atingir a 5FN, a tabela deve ser decomposta nas projeções que expressam a dependência de junção problemática, de modo que as dependências restantes nela sejam implicadas apenas pelas chaves candidatas.

Gabarito: Errado

Dependências funcionais na prática

Tudo que estudamos nesta aula sobre dependências funcionais, formas normais e chaves foi embasado no “mundo teórico”, ou seja, sem as verificarmos como de fato ocorrem em bancos de dados reais.

Pois bem, para materializarmos isso tudo na prática em um banco de dados, utilizamos os comandos PRIMARY KEY e UNIQUE da linguagem SQL.

Isso porque são justamente esses comandos que definem as chaves que uma tabela possui (sendo PRIMARY KEY para chave primária, e UNIQUE para as candidatas), nos permitindo avaliar se as dependências funcionais do negócio foram de fato bem implementadas nessas tabelas.

Para exemplificar, vamos supor o seguinte comando SQL.

```
CREATE TABLE Funcionario (  
  CPF      CHAR(11)  PRIMARY KEY,  
  RG       CHAR(9)   UNIQUE,  
  nome     VARCHAR(50)  
);
```

Dessa forma, só de olharmos para o código SQL, conseguimos perceber que CPF se trata de uma chave primária, então a tabela respeita a dependência funcional “CPF → {CPF, RG, nome}”.

Além disso, RG também se trata de uma chave (candidata, no caso, por ser UNIQUE), então sabemos que a dependência funcional “RG → {CPF, RG, nome}” também se sustenta.



(FGV/TJ AP/Tecnologia da Informação Desenvolvimento de Sistemas/2024) Considere uma tabela relacional criada a partir do script SQL a seguir.

```
create table xpto (  
  a int primary key,  
  b int not null,  
  c int not null,  
  d int not null)
```

De acordo com as formas normais das tabelas relacionais, o conjunto de dependências funcionais que deve necessariamente ser verificado é:

- a) a → b, a → c, a → d;
- b) a → b, b → c, c → d;
- c) a → b, c, d;
- d) b → a, c → a, d → a;
- e) b, c, d → a.

Comentários:

Como "a" é uma PRIMARY KEY, então se trata de uma chave. Dessa forma, o projeto de banco de dados obedece necessariamente a seguinte dependência funcional: $a \rightarrow \{a, b, c, d\}$.

Da decomposição, temos que

$a \rightarrow a$;

$a \rightarrow b$;

$a \rightarrow c$; e

$a \rightarrow d$.

Como $a \rightarrow a$ se trata da dependência trivial, podemos simplesmente ignorá-la.

Gabarito: A

Além disso, também podemos perceber quais Dependências Funcionais se sustentam olhando diretamente para os registros de uma tabela de banco de dados.

Para conseguirmos isso, basta seguirmos uma regra simples: **uma DF " $X \rightarrow Y$ " vale na instância se, sempre que duas linhas coincidem em X, também coincidem em Y.**

Para exemplificar, vamos supor a tabela a seguir.

CPF	nome	idade
1	João	20
2	Maria	30
2	Maria	40

Dessa forma, para essas instâncias, podemos verificar se cada dependência funcional " $X \rightarrow Y$ " possível entre os atributos "CPF", "nome" e "idade" é válida fazendo uma pergunta simples: **"Existe algum par de linhas que tem o mesmo valor em X mas valores diferentes em Y?"**

Se a resposta for "**sim**", então a DF **não vale** (achamos uma contradição). Já se "**não**", então a DF **vale**.

Então, testando todas possíveis DFs, temos:

- **CPF $\rightarrow$ Nome:** há CPF repetido nas linhas 2 e 3 (CPF = 2). O nome nessas linhas é Maria e Maria. Ou seja, sempre o mesmo CPF determina o mesmo nome. Então, não há contradição, e **a DF é válida**.
- **CPF $\rightarrow$ Idade:** há CPF repetido nas linhas 2 e 3 (CPF = 2), porém as respectivas idades nessas linhas são 30 e 40. Ou seja, são diferentes, há contradição! Então, saber que CPF = 2 não é suficiente para saber a idade, pois pode ser 30 ou 40. **A DF não é válida**.
- **Nome $\rightarrow$ CPF:** Maria aparece nas linhas 2 e 3 e o CPF nessas linhas vale 2 e 2. Ou seja, iguais, e **a DF é válida**.

- **Nome** → **Idade**: Maria aparece nas linhas 2 e 3, porém a idade vale 30 e 40. Como os valores do dependente são diferentes para o mesmo determinante, **a DF não vale**.
- **Idade** → **CPF**: Todos os valores de Idade são distintos (20, 30, 40), ou seja, não há repetição, não sendo possível haver contradição. **A DF é válida**.
- **Idade** → **Nome**: Mesma lógica, os valores de idade são todos distintos, sendo impossível haver contradição. **A DF é válida**.

Desnormalização

Na prática, dizemos que uma tabela está normalizada quando chega à 3FN, porque, nesse estado, as principais anomalias de inserção, exclusão e atualização encontram-se resolvidas.

Porém, por mais que o processo de normalização seja bem importante pelos problemas que evita, esse não é o único objetivo do design de um banco de dados.

Isso porque, quanto mais normalizamos, mais tabelas criamos no nosso banco. Assim, embora ganhamos por um lado, por ter dados menos redundantes e anômalos, perdemos por outro, por termos a informação "fragmentada" por diversas tabelas.

É que mais tabelas significam várias junções (em SQL, "JOINS") para montar uma informação completa, operação que pode resultar em consumo elevado de tempo e recursos.

Então, em situações muito específicas, como sistemas com bilhões de registros, relatórios extremamente complexos ou requisitos de velocidade críticos, pode ser vantajoso **desnormalizarmos** intencionalmente nossos dados. Ou seja, podemos **aceitar alguma redundância em troca de velocidade**.

Tipo de Desnormalização	Explicação
Dado redundante	Guardamos a mesma informação de várias tabelas em uma tabela desnormalizada , evitando junções em múltiplas tabelas. Por o dado existir em dois lugares simultaneamente (na tabela de origem e na tabela desnormalizada), a responsabilidade de manter os dois sincronizados passa a ser do sistema.
Dado derivado	Guardamos um atributo calculado (ou inferido) a partir de outro atributo já existente na na tabela . Assim, em vez de calcular o valor sempre que necessário, o computamos uma única vez e salvamos, trocando custo de processamento por custo de armazenamento.

Tipo de Desnormalização	Explicação
Dado pré-agregado	É um caso específico de dado derivado, onde o valor armazenado é resultado de uma operação de agregação, como soma, média ou contagem, que normalmente exigiria consultar e processar múltiplos registros de outras tabelas. O valor é calculado antecipadamente e atualizado sempre que os dados que o originam são modificados.
Tabela Temporária para Relatório	Ocorre quando a estrutura normalizada do banco torna impossível ou impraticável gerar um determinado relatório diretamente via consulta SQL. Assim, criamos uma tabela temporária desnormalizada exclusivamente para montar esse relatório, sendo descartada logo após o uso (portanto, não há preocupação com sincronização ou consistência).



(CEBRASPE (CESPE)/FUNPRESPE/EXE/Estatística Ciências de Dados e Inovação/2025) Na criação de esquemas de banco de dados eficientes, a normalização excessiva pode levar a um desempenho ruim em consultas complexas, e, por isso, deve ser sempre evitada.

Comentários:

A afirmação está errada porque dizer que a normalização "deve ser sempre evitada" é um exagero. A normalização traz diversos benefícios, como prevenção de anomalias de dados, redução de redundância não intencional, economia nos custos de armazenamento e recuperação mais rápida de dados. É verdade que existe o conceito de desnormalização, o que mostra que em alguns casos pode ser interessante abrir mão de parte da normalização por questões de desempenho. Porém, isso não significa que a normalização "deve ser sempre evitada". Ela é, na verdade, um processo fundamental para garantir que os dados estejam bem organizados e livres de anomalias de inserção, exclusão e atualização. O erro da questão está justamente nesse "sempre evitada", pois a normalização é essencial e seus benefícios são amplamente reconhecidos, cabendo ao projetista avaliar caso a caso quando eventualmente desnormalizar.

Gabarito: Errado

(FUNDATEC/PROCERGS/Administração de Dados/2023) Qual é a diferença entre normalização e desnormalização de dados?

a) Normalização é o processo de otimização da estrutura dos dados para evitar redundâncias e inconsistências, enquanto a desnormalização é o processo de adicionar redundâncias propositalmente para melhorar o desempenho de consultas frequentes.

- b) Normalização é o processo de aumentar a redundância de dados para aumentar a segurança, enquanto a desnormalização é o processo de remover a redundância para economizar espaço de armazenamento.
- c) Normalização é o processo de agrupar dados relacionados em uma única tabela, enquanto a desnormalização é o processo de dividir dados relacionados em várias tabelas para melhorar a escalabilidade.
- d) Normalização é o processo de modificar a estrutura de dados para atender a requisitos específicos de aplicativos, enquanto a desnormalização é o processo de restaurar dados em uma estrutura original após um processo de normalização.
- e) Normalização e desnormalização são sinônimos que descrevem o processo de modificação de dados para atender a diferentes requisitos de aplicativos.

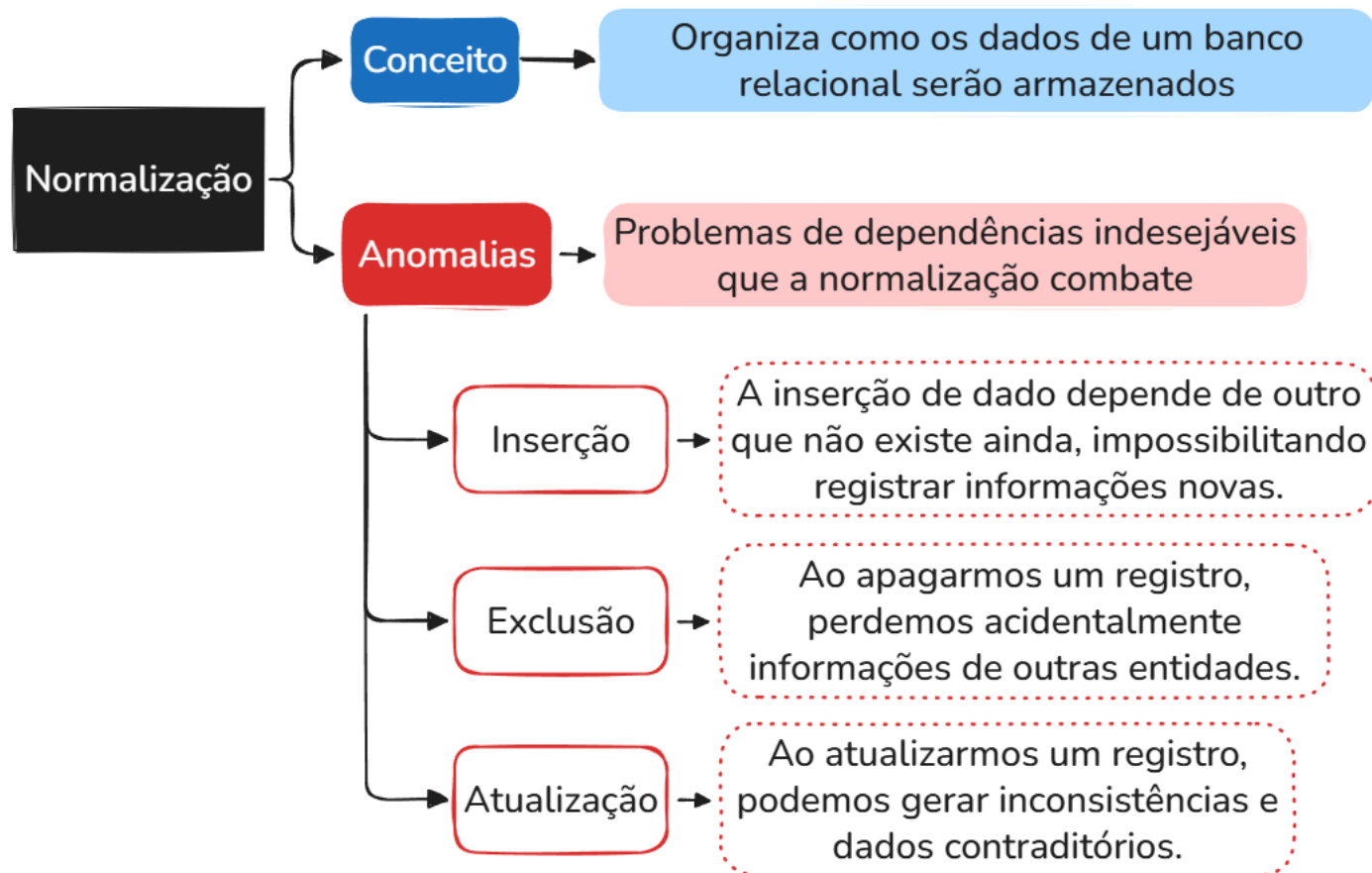
Comentários:

- a) **Correta.** Normalização organiza os dados aplicando formas normais para reduzir redundâncias e prevenir anomalias (inserção, exclusão e atualização). Já a desnormalização aceita intencionalmente alguma redundância para ganhar velocidade nas consultas, evitando junções complexas entre muitas tabelas.
- b) **Errada.** Normalização reduz redundância, não aumenta. E seu objetivo não é aumentar segurança, mas sim organizar os dados e garantir integridade.
- c) **Errada.** Na verdade é o contrário: a normalização decompõe tabelas em tabelas menores para eliminar dependências problemáticas, enquanto a desnormalização junta informações em menos tabelas para melhorar desempenho.
- d) **Errada.** Normalização não é sobre atender requisitos específicos de aplicativos, mas sim sobre organizar dados para evitar anomalias e redundâncias. E desnormalização não é "restaurar dados à estrutura original", é aceitar redundância de propósito para ganhar desempenho.
- e) **Errada.** Normalização e desnormalização são processos opostos. Normalização elimina redundâncias aplicando formas normais, enquanto desnormalização reintroduz redundâncias intencionalmente em troca de velocidade.

Gabarito: A

RESUMO

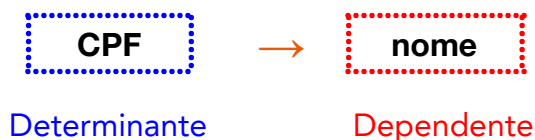
Introdução



Benefício	Explicação
Prevenção de Anomalias de Dados	Com a normalização, a alteração de um banco de dados torna-se um processo mais confiável, sendo menos propenso a erros.
Redução da Redundância Não Intencional	Dados duplicados por acidente, fruto de design ruim, geram inconsistências e desperdícios. Com a normalização, combatemos esse tipo de problema.
Economia nos Custos de Armazenamento	Cada dado duplicado ocupa espaço físico real, seja em disco local ou em servidores na nuvem. Em ambientes de nuvem como AWS, Google Cloud ou Azure, onde pagamos exatamente pelo que usamos, esse benefício se traduz em economia financeira direta.

Benefício	Explicação
Recuperação Mais Rápida de Dados	Quando um banco de dados tem menos dados redundantes, as consultas se tornam mais eficientes, exigindo menor poder de processamento.

Uma **Dependência Funcional** (DF) existe quando os **valores de algumas colunas definem obrigatoriamente os valores de outras**.



Reflexividade

Um conjunto de atributos sempre determina suas partes

$\{CPF, nome\} \Rightarrow \{CPF, nome\} \rightarrow CPF$
 $\{CPF, nome\} \Rightarrow \{CPF, nome\} \rightarrow nome$
 $\{CPF, nome\} \Rightarrow \{CPF, nome\} \rightarrow \{CPF, nome\}$

Augmentação

Podemos adicionar atributo extra à toda dependência sem a invalidar

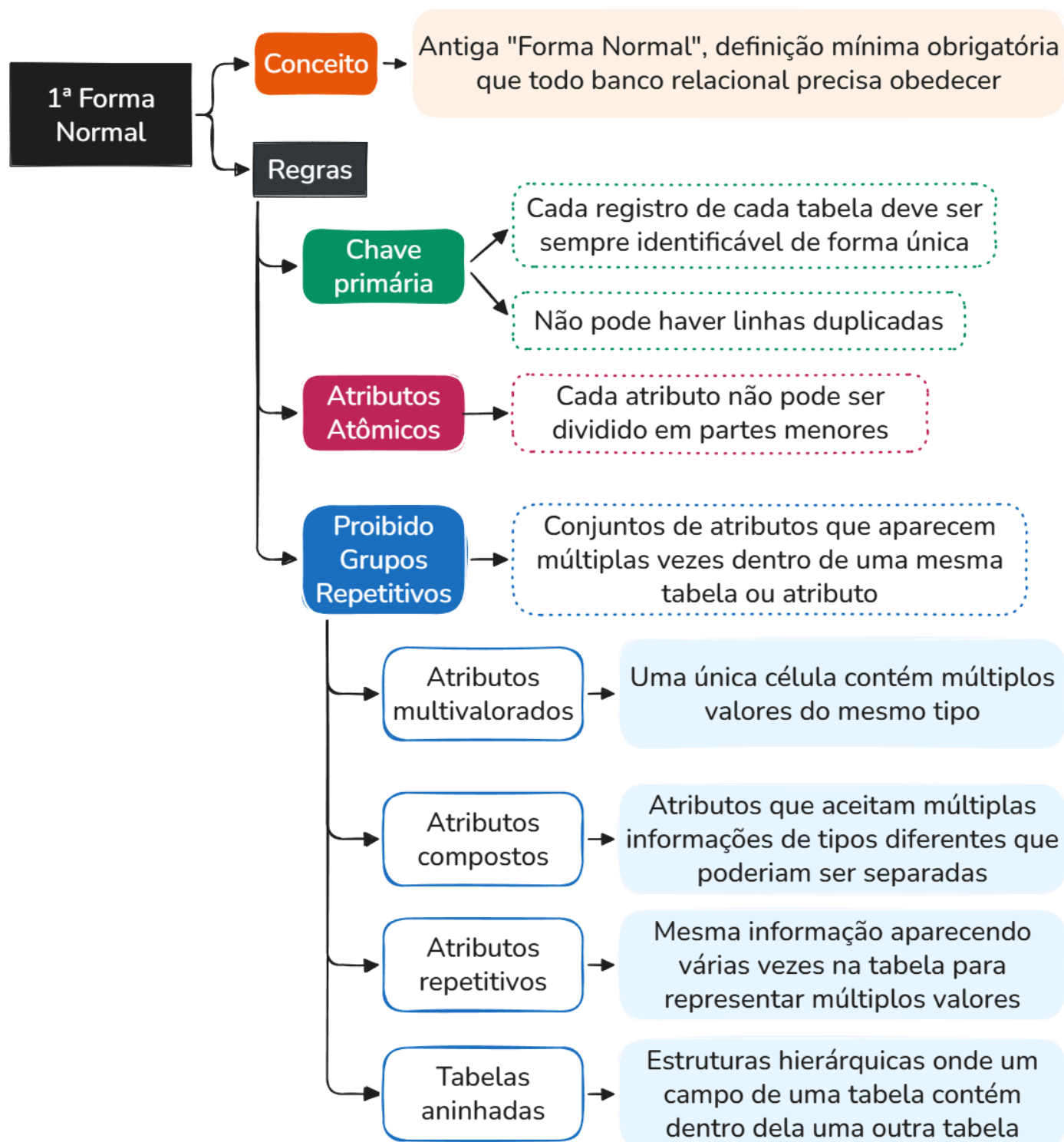
$CPF \rightarrow nome \Rightarrow \{CPF, sobrenome\} \rightarrow \{nome, sobrenome\}$

Transitividade

Podemos encadear duas dependências se tiverem atributos iguais no determinante de uma e determinado da outra

$CPF \rightarrow CEP$
 $CEP \rightarrow endereço \Rightarrow CPF \rightarrow endereço$

Formas Normais



Pessoas		
CPF	nome	telefones
111	Maria B.	9871-4751, 9471-1274, 9544-4756
222	Maria	9841-5151
333	Joana	9157-7474, 9334-2798

Atributo Multivalorado

Pessoas		
CPF	nome	endereço
111	Maria B.	Rua um, nº 57, apto 3, São Luís - MA
222	Maria	Rua couve, nº 44, São Gonçalo - RJ
333	Joana	Rua pássaros, nº 101, Inocência - MS

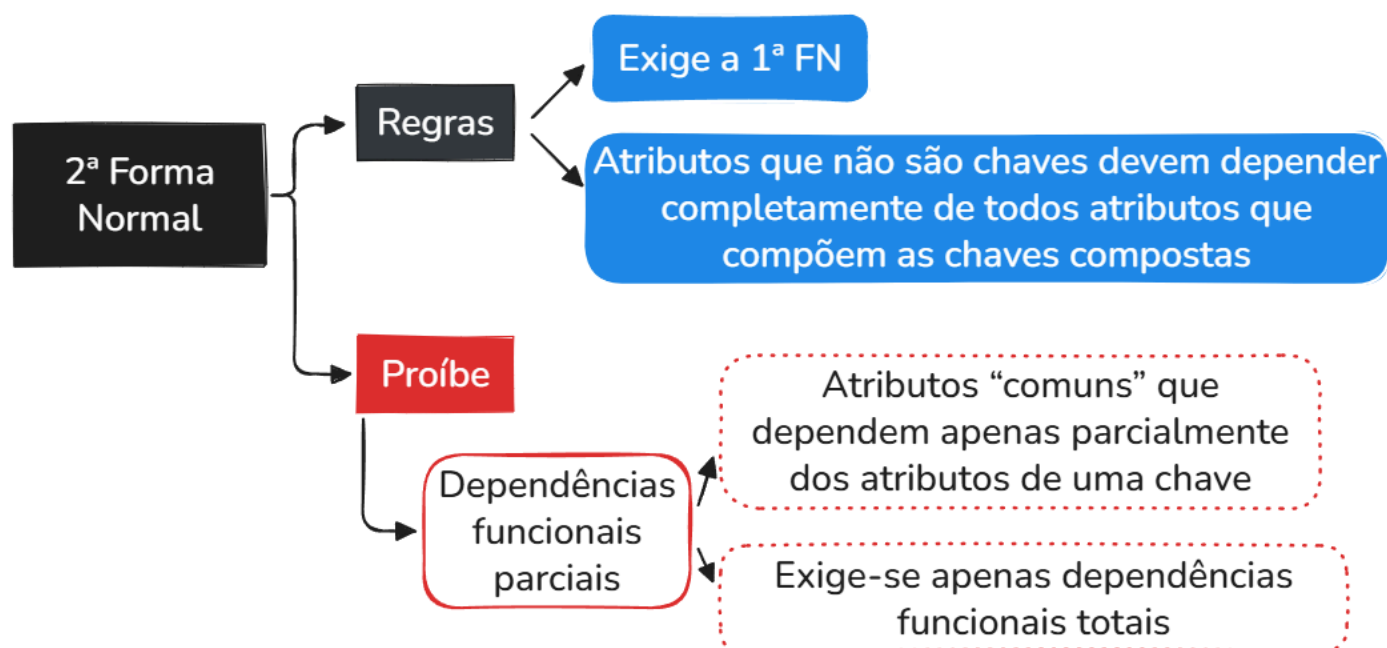
Atributo Composto

Pessoas				
CPF	nome	telefone1	telefone2	telefone3
111	Maria B.	9871-4751	9471-1274	9544-4756
222	Maria	9841-5151	Null	Null
333	Joana	9157-7474	9334-2798	Null

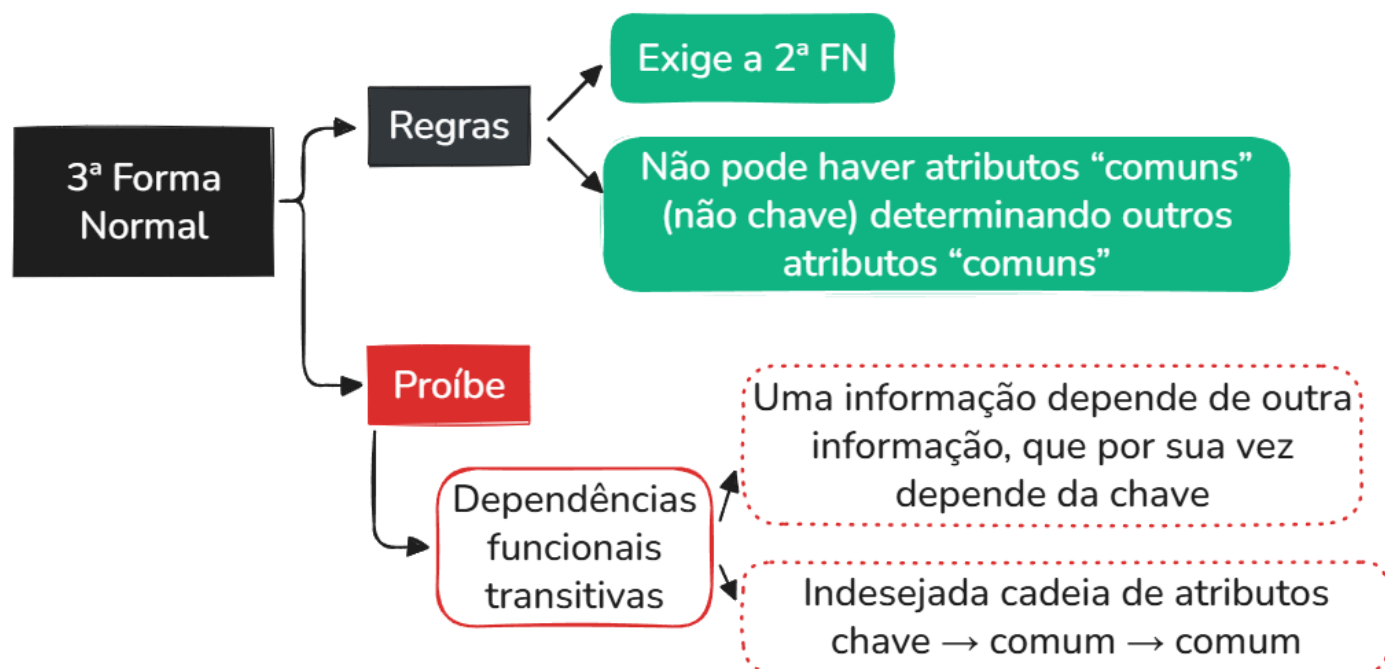
Atributos Repetitivos

Pessoas							
CPF	nome	telefones					
111	Maria B.	<table><tr><th>Telefones</th></tr><tr><th>telefone</th></tr><tr><td>9871-4751</td></tr><tr><td>9841-5151</td></tr><tr><td>9157-7474</td></tr></table>	Telefones	telefone	9871-4751	9841-5151	9157-7474
Telefones							
telefone							
9871-4751							
9841-5151							
9157-7474							

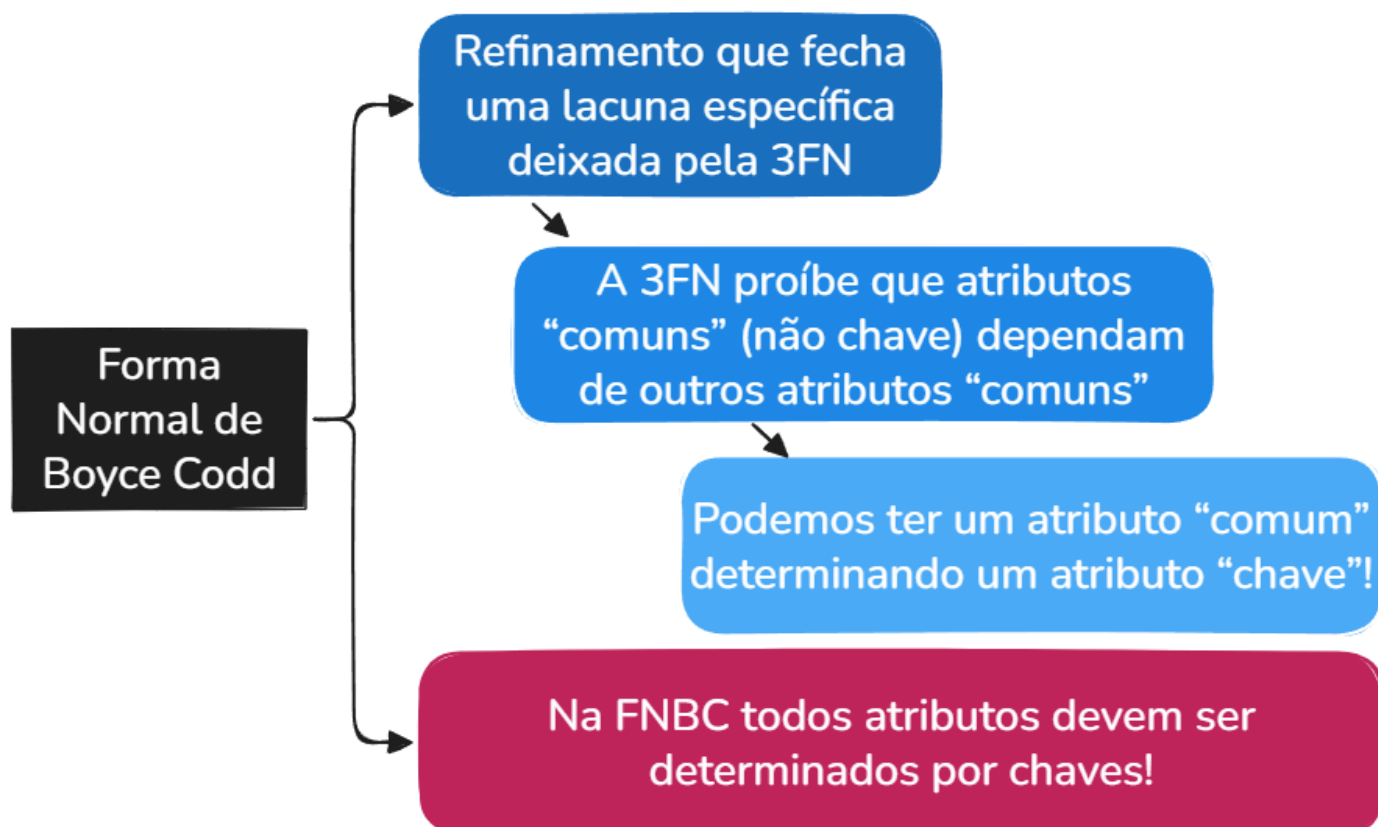
Tabela Aninhada



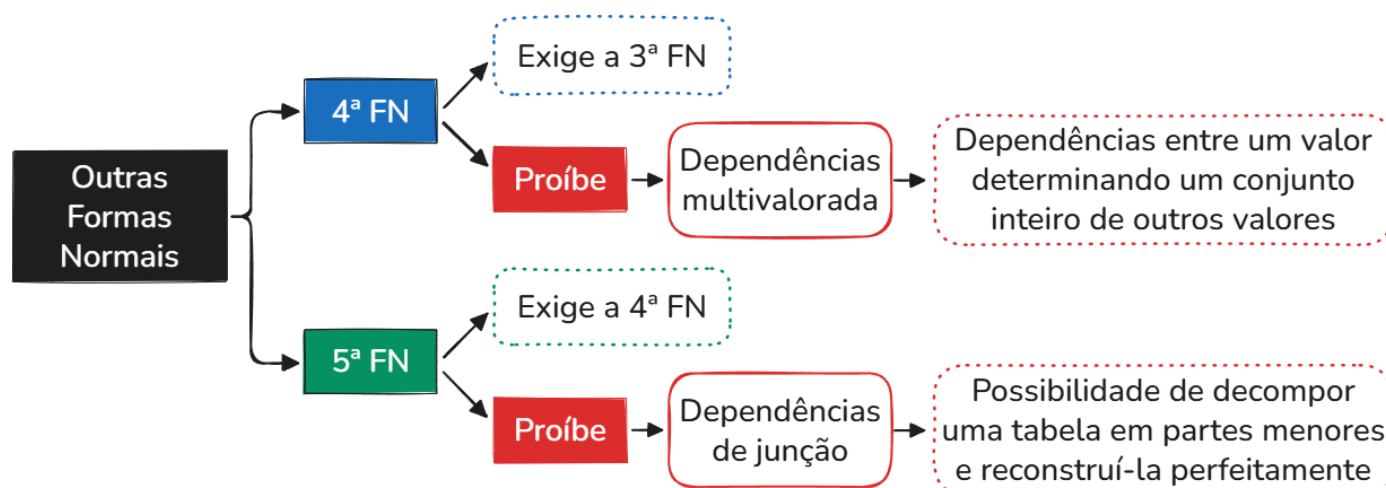
Funcionários e Dependentes			
CPF funcionário	CPF dependente	nome funcionário	nome dependente
111	999	Josefa	Mario
222	888	Maria	João
333	777	Joana	José



Funcionários e Departamentos			
<u>id funcionário</u>	nome	id departamento	nome departamento
111	Josefa	1	Exatas
222	Maria	2	Biológicas
333	Joana	3	Humanas



Caso	3ª FN	FNBC
comum → comum	✗	✗
comum → chave	✓	✗
chave → comum	✓	✓



QUESTÕES COMENTADAS - FGV



1. (FGV/AMAZUL/Técnico de Informática/2026) No processo de projeto lógico de bancos de dados relacionais, a normalização organiza os atributos das relações de forma a eliminar redundâncias indesejadas e anomalias de atualização.

Dentro desse encadeamento, a forma normal que tem sua definição fundamentada direta e exclusivamente no conceito de dependência funcional total é a

- a) primeira forma normal (1FN).
- b) segunda forma normal (2FN).
- c) terceira forma normal (3FN).
- d) forma normal Boyce-Codd (FNBC).
- e) quarta forma normal (4FN).

Comentários:

a) Errada. A 1FN se fundamenta em exigências como chave primária, atomicidade dos atributos e ausência de grupos repetitivos, não no conceito de dependência funcional total.

b) Correta. A 2FN exige que todos os atributos que não são chave dependam completamente de todos os atributos que compõem a chave, ou seja, proíbe dependências parciais. Isso é exatamente o conceito de dependência funcional total.

c) Errada. A 3FN se fundamenta no conceito de dependência transitiva, proibindo que atributos comuns determinem outros atributos comuns, e não no conceito de dependência funcional total.

d) Errada. A FNBC se fundamenta na ideia de que apenas chaves podem determinar outros atributos, fechando uma brecha da 3FN. Seu foco não é a dependência funcional total.

e) Errada. A 4FN se fundamenta no conceito de dependência multivalorada, que trata de um valor determinando um conjunto inteiro de outros valores, algo diferente de dependência funcional total.

Gabarito: B

2. (FGV/AAD (TCE,RR)/TCE RR/Tecnologia da Informação/Banco de Dados/2025) O conceito de dependência funcional é fundamental na análise de esquemas relacionais, sendo uma ferramenta básica para medir a adequação dos agrupamentos de atributos. Dependências funcionais podem ser utilizadas para descrever melhor um esquema de relação ao especificar restrições que devem ser mantidas entre seus atributos.

As dependências funcionais problemáticas podem ser eliminadas por meio do processo de

normalização, buscando alcançar as várias formas normais.

As formas normais que buscam eliminar novos conceitos de dependência funcional que são extensões e especializações do conceito original são

- a) 1FN e 5FN.
- b) 2FN e 3FN.
- c) 3FN e Forma Normal de Boyce-Codd (FNBC).
- d) 4FN e 5FN.
- e) 3FN e 4FN.

Comentários:

a) Errada. A 1FN trata de atomicidade e chave primária, não de dependências funcionais, e a 5FN trata de dependência de junção, que é uma extensão, mas a 1FN não se encaixa.

b) Errada. A 2FN e a 3FN trabalham com o conceito original de dependência funcional (parcial e transitiva), não com extensões ou especializações dele.

c) Errada. A 3FN e a FNBC também trabalham com o conceito original de dependência funcional, apenas refinando as regras sobre quem pode determinar quem.

d) Correta. A 4FN introduz o conceito de dependência multivalorada, que é um valor determinando um conjunto inteiro de outros valores, e a 5FN introduz o conceito de dependência de junção, que trata de decomposição e reconstrução de tabelas. Ambos são conceitos novos que vão além da dependência funcional original, sendo extensões e especializações dela.

e) Errada. A 3FN trabalha com dependência transitiva, que ainda faz parte do conceito original de dependência funcional, não sendo uma extensão ou especialização nova.

Gabarito: D

3. (FGV/AAD (TCE,RR)/TCE RR/Tecnologia da Informação/Banco de Dados/2025) Considerando o processo de normalização de um banco de dados relacional, avalie as afirmativas a seguir.

I. Um dos objetivos da normalização é minimizar redundância.

II. A normalização oferece uma estrutura formal para analisar esquemas de relação com base em suas chaves.

III. Desnormalização é o processo de armazenar a junção de relações na forma normal mais baixa como uma relação básica.

Está correto o que se afirma em

- a) I, apenas.
- b) II, apenas.
- c) I e II, apenas.
- d) I e III, apenas.
- e) II e III, apenas.

Comentários:

A afirmativa I está certa porque um dos benefícios da normalização é justamente a redução da redundância não intencional, ou seja, dados duplicados por acidente que geram inconsistências e desperdícios.

A afirmativa II está certa porque a normalização aplica sucessivamente formas normais que analisam as tabelas com base em suas chaves e nas dependências funcionais entre os atributos, verificando, por exemplo, se atributos dependem completamente da chave (2FN), se há dependências transitivas (3FN) ou se apenas chaves determinam outros atributos (FNBC).

A afirmativa III está errada porque desnormalização não é definida como armazenar a junção de relações na forma normal "mais baixa"; o conceito de desnormalização envolve abrir mão de parte da normalização por questões de desempenho, e não se trata de reduzir tudo à forma normal mais baixa.

Gabarito: C

4. (FGV/TJ TRT24/TRT 24/Apoio Especializado/Tecnologia da Informação/2025) Durante a instalação de um SGBD, o administrador de dados decidiu aplicar princípios de normalização. Assinale a opção que indica a finalidade de adotar tais princípios.

- a) Automatizar a criação de índices em todas as colunas, aumentando a velocidade das consultas.
- b) Reduzir redundâncias, otimizando a manutenção do banco de dados.
- c) Garantir alocação dinâmica de memória no servidor, acelerando o acesso aos dados.
- d) Otimizar a estrutura, reduzindo a frequência de backups incrementais.
- e) Armazenar todas as chaves estrangeiras em uma única tabela, simplificando as relações entre as tabelas.

Comentários:

a) Errada. A normalização não tem nada a ver com criação automática de índices. Ela organiza os dados em tabelas para evitar problemas de estrutura.

b) Correta. A normalização serve justamente para reduzir redundâncias não intencionais (dados duplicados por acidente) e prevenir anomalias de inserção, exclusão e atualização, o que torna a manutenção do banco de dados mais confiável e eficiente.

c) Errada. Alocação de memória no servidor é um assunto de infraestrutura, não tem relação com normalização. A normalização trata da organização dos dados nas tabelas.

d) Errada. A normalização não tem relação com a frequência de backups. Seu objetivo é organizar os dados para evitar redundâncias e anomalias.

e) Errada. A normalização não propõe armazenar chaves estrangeiras em uma única tabela. Pelo contrário, o processo de normalização muitas vezes decompõe tabelas em tabelas menores e mais organizadas, cada uma com suas próprias chaves.

Gabarito: B

5. (FGV/AJ (TJ AP)/TJ AP/Apoio Especializado/Tecnologia da Informação Banco de Dados/2024) Considere uma tabela relacional com atributos (colunas) A, B, C, D e E, na qual as

dependências funcionais conhecidas são:

$A \rightarrow B$

$A \rightarrow C$

$A \rightarrow D$

$D \rightarrow E$

O comando de criação dessa tabela é mostrado a seguir.

```
create table X(A int, B int, C int, D int, E int,  
constraint K1 unique (A),  
constraint K2 unique (D)  
)
```

De acordo com as formas normais Primeira, Segunda, Terceira e Boyce-Codd, a dependência funcional adicional necessária e suficiente para a validade desse esquema é:

a) $A \rightarrow E$;

b) $A, D \rightarrow E$;

c) $D \rightarrow A$;

d) $D \rightarrow B, C, D$;

e) $E \rightarrow A$.

Comentários:

Na FNBC todo determinante das dependências funcionais deve ser chave. Assim, precisamos deduzir as DFs das chaves fornecidas no SQL.

Assim, do SQL, extraímos que A e D são chaves (graças ao comando UNIQUE). Como A é chave, então $A \rightarrow \{B, C, D, E\}$. Como D é chave, então $D \rightarrow \{A, B, C, E\}$ (ignoramos as dependências triviais $A \rightarrow A$ e $D \rightarrow D$).

Assim sendo, da decomposição, obtemos que o projeto de banco de dados respeita as DFs a seguir, bastando apenas essas para estar na FNBC.

$A \rightarrow B$

$A \rightarrow C$

$A \rightarrow D$

$A \rightarrow E$

$D \rightarrow A$

$D \rightarrow B$

$D \rightarrow C$

$D \rightarrow E$

Pois bem, agora precisamos analisar as DFs que o examinador forneceu para verificar quais faltam para obtermos essas exatas DFs que acabamos de deduzir.

$A \rightarrow B$

$A \rightarrow C$

$A \rightarrow D$

$D \rightarrow E$

Da transitividade, como $A \rightarrow D$ e $D \rightarrow E$, então $A \rightarrow E$.

Nesse momento já deduzimos todas DFs que exigem "A" como chave.

Porém, não conseguimos deduzir mais nenhuma DF de "D" como chave.

Então, para completar todas DFs que faltam com "D" como chave, basta adicionarmos a dependência " $D \rightarrow A$ ".

Isso porque, pela transitividade dela com as outras DFs que tem "A como chave", obtemos todas as outras dependências funcionais de "D", vejamos:

$D \rightarrow A$ e $A \rightarrow B$, então $D \rightarrow B$

$D \rightarrow A$ e $A \rightarrow C$, então $D \rightarrow C$

$D \rightarrow A$ e $A \rightarrow E$, então $D \rightarrow E$

Gabarito: C

6. (FGV/Tec NS (TJ MS)/TJ MS/Analista de Sistemas Computacionais/Analista de Banco de Dados/2024) A normalização de dados é o processo de analisar os esquemas de relações com base nas dependências funcionais de seus atributos e chaves primárias, envolvendo uma série de Formas Normais (FN).

Observe o esquema da tabela "Andamento_Processo", a seguir.

IDAND	IDPRC	DATA	DESCRICAO	ADV_RESPONSAVEL
1	1	2024-01-11	Petição Inicial	Maria
2	1	2024-01-25	Audiência	Ana
3	2	2024-01-30	Interrogatório	Luiza

A tabela "Andamento_Processo" possui chave primária composta, atributos com valores atômicos e ausências de dependências transitivas entre os atributos não chave.

Com base nisso, é correto afirmar que "Andamento_Processo" está normalizada na FN:

- a) 1FN;
- b) 2FN;
- c) 3FN;
- d) 4FN;
- e) 5FN.

Comentários:

a) **Errada.** A tabela está na 1FN (tem chave primária e atributos atômicos), mas ela vai além disso, pois também cumpre requisitos de formas normais superiores.

b) **Errada.** A tabela também está na 2FN, já que possui chave composta e não foram indicadas dependências parciais, mas ela vai além disso.

c) **Correta.** A tabela tem chave primária composta, atributos atômicos (cumpre a 1FN), não há dependências parciais indicadas (cumpre a 2FN) e o enunciado afirma expressamente que não há dependências transitivas entre atributos não chave (cumpre a 3FN). Lembre que cada forma normal herda as regras das anteriores, então uma tabela na 3FN automaticamente está na 1FN e na 2FN. Como a maior forma normal atendida com as informações dadas é a 3FN, essa é a resposta.

d) **Errada.** Para estar na 4FN seria necessário analisar dependências multivaloradas, algo que o enunciado não menciona, então não há como afirmar que a tabela chega a esse nível.

e) **Errada.** Para estar na 5FN seria necessário analisar dependências de junção, algo que o enunciado também não menciona.

Gabarito: C

7. (FGV/Tec NS (TJ MS)/TJ MS/Analista de Sistemas Computacionais/Analista de Sistemas/2024) A normalização de dados é o processo de analisar os esquemas de relações com base nas dependências funcionais de seus atributos e chaves primárias, envolvendo uma série de Formas Normais (FN). Observe o esquema da tabela "Pedido", a seguir.

ID_PEDIDO	ID_CLIENTE	DATA	NOME_CLIENTE
1	2	16/04/2024	Luiza
2	3	16/04/2024	Júlia

A tabela "Pedido" possui chave primária composta (ID_PEDIDO, ID_CLIENTE), atributos com valores atômicos e dependência parcial de coluna não chave (NOME_CLIENTE) com parte da chave primária (ID_CLIENTE).

Com base nisso, "Pedido" está normalizada na FN:

- a) 1FN;
- b) 2FN;
- c) 3FN;
- d) 4FN;
- e) 5FN.

Comentários:

a) **Correta.** A tabela possui chave primária e todos os atributos têm valores atômicos, então ela cumpre as regras da 1FN. Porém, como existe uma dependência parcial (NOME_CLIENTE depende apenas de ID_CLIENTE, que é só uma parte da chave composta), ela viola a 2FN. Logo, ela está normalizada apenas na 1FN.

b) Errada. Para estar na 2FN, todos os atributos não chave precisam depender completamente de toda a chave composta. Como NOME_CLIENTE depende apenas de ID_CLIENTE (parte da chave), existe dependência parcial, o que viola a 2FN.

c) Errada. A 3FN exige que a tabela já esteja na 2FN, o que não acontece aqui por causa da dependência parcial.

d) Errada. A 4FN exige que todas as formas normais anteriores sejam cumpridas, e essa tabela nem mesmo está na 2FN.

e) Errada. A 5FN exige que todas as formas normais anteriores sejam cumpridas, e essa tabela nem mesmo está na 2FN.

Gabarito: A

8. (FGV/Prof NS (SES MT)/SES MT/Analista de Sistemas/Administrador de Banco de Dados ou Infraestrutura e O&M/2024) Existem diversas formas normais que podem ser utilizadas na normalização de esquemas relacionais. A forma normal Boyce- Codd (FNBC) foi proposta para ser uma simplificação, mas descobriu-se que ela é mais rigorosa que a forma normal que ela pretendia simplificar.

A forma normal que a FNBC pretendia simplificar é a

- a) 1FN.
- b) 2FN.
- c) 3FN.
- d) 4FN.

Comentários:

a) Errada. A 1FN trata de chave primária e atomicidade dos atributos, não tem relação direta com a FNBC.

b) Errada. A 2FN trata de dependência funcional total, proibindo dependências parciais, e não é o alvo de refinamento da FNBC.

c) Correta. A FNBC é frequentemente chamada de 3.5FN justamente porque vai além da 3FN, fechando uma brecha que ela deixava: a 3FN proíbe que atributos comuns determinem outros atributos comuns, mas permite que um atributo comum determine um atributo de chave. A FNBC corrige isso exigindo que apenas chaves possam determinar outros atributos.

d) Errada. A 4FN vem depois da FNBC e trata de dependências multivaloradas, que é um conceito diferente.

Gabarito: C

9. (FGV/STN/Tecnologia da Informação/Operação e Infraestrutura/2024) Com referência à normalização de bancos de dados, considere uma tabela relacional T, com colunas C, H, N, P, na qual as dependências funcionais identificadas são exibidas a seguir.

C-> H

C -> P

C -> N

$P \rightarrow C$

O comando SQL utilizado na criação dessa tabela é exibido a seguir.

```
create table T ( C int not null unique,  
H int null unique,  
P int null unique,  
N int not null  
)
```

Assinale a dependência funcional adicional necessária para que o esquema acima esteja de acordo com a forma normal Boyce-Codd.

- a) $C, H, P \rightarrow N$
- b) $H \rightarrow P$
- c) $N \rightarrow P$
- d) $N \rightarrow H$
- e) $P, H \rightarrow C$

Comentários:

Na forma normal Boyce-Codd, apenas as chaves da tabela podem ser determinantes. Da tabela "T", temos que as chaves são C, H e P, então podemos deduzir que $C \rightarrow \{C, H, P, N\}$, $H \rightarrow \{C, H, P, N\}$ e $P \rightarrow \{C, H, P, N\}$.

Ou seja, precisamos chegar nessas 3 dependências funcionais analisando as fornecidas pelo enunciado.

Assim, das dependências funcionais fornecidas, temos que $C \rightarrow H$, $C \rightarrow P$ e $C \rightarrow N$, então $C \rightarrow \{C, H, P, N\}$ (considerando também a dependência trivial $C \rightarrow C$). Aqui percebemos que as DFs com "C" como determinante foram dadas.

Além disso, como $P \rightarrow C$, da transitividade com $C \rightarrow \{C, H, P, N\}$, temos que $P \rightarrow \{C, H, P, N\}$. Ou seja, as DFs com "P" como determinante foram dadas.

Portanto, só faltam as dependências com "H". Para isso, basta " $H \rightarrow P$ ", já que com a transitividade com $P \rightarrow \{C, H, P, N\}$, temos que $H \rightarrow \{C, H, P, N\}$.

Gabarito: B

10. (FGV/TJ (TRF 1ª Região)/TRF 1/Apoio Especializado/Tecnologia da Informação/2024) Roberta, funcionária da ORG_A, está utilizando instruções em SQL para fazer inclusões, alterações e exclusões de registros em um banco de dados que contém as informações de funcionários que participaram de determinados treinamentos promovidos pela ORG_A. Roberta notou algumas anomalias de exclusão, pois, ao comandar a instrução SQL para excluir um funcionário desligado da organização, alguns treinamentos desapareceram.

Considerando que as instruções em SQL executadas por Roberta estão sintática e semanticamente corretas, a modelagem do banco de dados possui problemas relacionados à existência de:

- a) dependências multivaloradas de junção;
- b) chave primária com mais de um campo não atômico;
- c) atributos com dependências não funcionais das chaves candidatas;

- d) dependências funcionais entre as chaves candidatas e a chave primária;
- e) dependências transitivas de atributos não chaves em relação à chave primária.

Comentários:

A anomalia de exclusão acontece quando, ao remover um registro, o banco elimina junto informações de outra entidade que deveriam continuar existindo. No caso, ao excluir um funcionário desligado, os treinamentos associados desaparecem. Isso ocorre porque dados de funcionários e treinamentos estão misturados na mesma tabela. Usualmente esse tipo de problema pode ser resolvido aplicando a 2FN ou a 3FN. Aqui, escolhe-se como solução aplicar a 3FN, porque é a única Forma Normal, dentre as duas, que podemos atingir pelas alternativas.

Gabarito: E

11. (FGV/AJ (TRF 1ª Região)/TRF 1/Apoio Especializado/Análise de Dados/2024) Na teoria do processo de normalização em bancos de dados relacionais, o conceito de chave é definido por “um conjunto de uma ou mais colunas, tal que não possa haver duas linhas, t_1 , t_2 , cujos valores presentes nessas colunas sejam idênticos em t_1 e t_2 ”.

Nesse contexto, considere uma tabela T, com colunas A, B, C e D, para as quais são válidas as seguintes dependências funcionais:

$A \rightarrow B$

$B \rightarrow C$

$B \rightarrow D$

$C \rightarrow B$

Supondo que, adicionalmente, a dependência funcional

$C \rightarrow A$

seja válida, é correto concluir que, obrigatoriamente, a tabela T deve ser:

- a) declarada com duas chaves distintas;
- b) declarada com três chaves distintas;
- c) declarada com uma chave apenas;
- d) particionada em duas tabelas;
- e) particionada em três tabelas.

Comentários:

Para ser uma chave, o atributo deve determinar todos os demais. Podemos derivar as seguintes dependências funcionais:

- Da transitividade, $A \rightarrow B$ e $B \rightarrow C$, então $A \rightarrow C$.
- Da transitividade, $A \rightarrow B$ e $B \rightarrow D$, então $A \rightarrow D$.

Portanto, A é chave, pois $A \rightarrow \{A, B, C, D\}$

- Da transitividade, $B \rightarrow C$ e $C \rightarrow A$, então $B \rightarrow A$.

Portanto, B é chave, pois $B \rightarrow \{A, B, C, D\}$.

- Da transitividade, $C \rightarrow B$ e $B \rightarrow \{A, B, C, D\}$, então $C \rightarrow \{A, B, C, D\}$.

Portanto, C também é chave.

Gabarito: B

12. (FGV/FTE (SEFAZ MT)/SEFAZ MT/2023) A normalização das estruturas de dados é uma etapa importante do processo de modelagem relacional para eliminar distorções ou anomalias no modelo.

Diz-se que uma tabela está na segunda forma normal, de acordo com as regras de normalização, se ela está na primeira forma normal e

a) as linhas da tabela são unívocas, sem chaves compostas e todos os seus atributos são atômicos.

b) as linhas não contêm itens repetitivos, atributos com valores nulos e nem multidimensionais.

c) cada uma das colunas não pertencentes à chave primária não for dependente parcialmente dessa chave.

d) contém apenas chaves externas e conecta-se a outras tabelas por múltiplas junções.

e) as colunas não pertencentes à chave primária ficam determinadas transitivamente por esta.

Comentários:

a) Errada. Isso descreve características da primeira forma normal (atomicidade, identificação única das linhas), não da segunda.

b) Errada. Mistura conceitos que não correspondem à definição da segunda forma normal. Itens repetitivos e atomicidade são questões da primeira forma normal, e "multidimensionais" não faz parte das regras de normalização aqui estudadas.

c) Correta. A segunda forma normal exige que todos os atributos que não são chave dependam completamente de toda a chave, ou seja, não pode haver dependência parcial. Dependência parcial é quando um atributo não chave depende de apenas uma parte da chave composta. Essa alternativa diz exatamente isso: nenhuma coluna fora da chave pode depender parcialmente dela.

d) Errada. Chaves externas e junções não têm relação com a definição da segunda forma normal.

e) Errada. Essa alternativa descreve a terceira forma normal, que é a que proíbe dependências transitivas, ou seja, quando um atributo comum determina outro atributo comum, criando uma cadeia do tipo "chave determina comum, que determina outro comum".

Gabarito: C

13. (FGV/DPE RS/Apoio Especializado (TI)/Banco de Dados/2023) A primeira forma normal, utilizada no processo de normalização de um banco de dados relacional, estabelece que:

a) não haja linhas duplicadas numa tabela;

b) não haja colunas idênticas numa tabela;

c) em nenhuma das linhas de uma tabela haja valores múltiplos armazenados na mesma coluna;

d) em qualquer tabela deve haver uma coluna que possa ser declarada como chave primária;

e) não haja transitividade de dependências funcionais entre as colunas.

Comentários:

a) Errada. Não ter linhas duplicadas é uma consequência de ter chave primária, mas não é a definição principal que a 1FN estabelece.

b) Errada. A proibição de colunas idênticas não é o que define a 1FN.

c) Correta. A 1FN exige a atomicidade dos atributos, ou seja, cada campo deve conter sempre um único valor indivisível. Isso significa que não podem existir valores múltiplos armazenados na mesma coluna, como por exemplo um campo "telefones" com vários telefones numa única célula.

d) Errada. Embora a 1FN também exija a existência de uma chave primária, não há necessidade de ser "uma coluna", visto que a chave pode ser composta (múltiplas colunas formando uma chave primária).

e) Errada. A proibição de dependências transitivas é regra da 3FN, não da 1FN.

Gabarito: C

14. (FGV/Pref Niterói/Tecnologia da Informação/2023) Dependências funcionais são usadas no processo de normalização de tabelas em projetos de bancos de dados relacionais e possuem propriedades que permitem a derivação de novas dependências. Considerando-se os atributos A, B, C de uma tabela relacional T, a derivação que NÃO é válida é:

a) $A, B, C \rightarrow A, B$;

b) se $A \rightarrow B$ e $A \rightarrow C$ então $A \rightarrow B, C$;

c) se $A \rightarrow B$ e $B \rightarrow C$ então $A \rightarrow C$;

d) se $A \rightarrow C$ então $A, B \rightarrow B, C$;

e) se $B \rightarrow A$ e $C \rightarrow A$ então $B \rightarrow C$.

Comentários:

a) Correta. $A, B, C \rightarrow A, B$ é uma aplicação direta da Reflexividade: se o lado direito é subconjunto do lado esquerdo, a dependência é verdadeira.

b) Correta. Se $A \rightarrow B$ e $A \rightarrow C$, então $A \rightarrow B, C$. Essa é a Regra da União, derivada dos axiomas de Armstrong. Quando um mesmo determinante gera duas dependências, podemos unir os dependentes no lado direito.

c) Correta. Se $A \rightarrow B$ e $B \rightarrow C$, então $A \rightarrow C$. Essa é a Transitividade, um dos três axiomas fundamentais de Armstrong. Se A determina B e B determina C, então A determina C indiretamente.

d) Correta. Se $A \rightarrow C$, então $A, B \rightarrow B, C$. Essa é a Regra do Aumento. Partindo de $A \rightarrow C$, adicionamos B a ambos os lados, obtendo $A, B \rightarrow C, B$.

e) Errada. Se $B \rightarrow A$ e $C \rightarrow A$, então $B \rightarrow C$. Essa derivação não é válida. O fato de B e C determinarem individualmente A não cria nenhuma relação funcional entre B e C.

Gabarito: E

15. (FGV/TJ (TJ SE)/TJ SE/Administrativa/Programação de Sistemas/2023) O processo de normalização de dados consiste em projetar relações normalizadas para organizar os dados em um banco de dados relacional e aumentar a:

a) anomalia de inserção;

b) anomalia de exclusão;

- c) integridade de dados;
- d) redundância de dados;
- e) anomalia de atualização.

Comentários:

a) Errada. A normalização serve justamente para prevenir anomalias de inserção, não para aumentá-las.

b) Errada. A normalização serve justamente para prevenir anomalias de exclusão, não para aumentá-las.

c) Correta. A normalização organiza os dados em um banco relacional para minimizar redundâncias e promover a integridade dos dados, permitindo que as restrições como chaves primárias, chaves estrangeiras e regras de unicidade protejam os dados de fato.

d) Errada. A normalização reduz a redundância não intencional, não a aumenta.

e) Errada. A normalização serve justamente para prevenir anomalias de atualização, não para aumentá-las.

Gabarito: C

16. (FGV/AJ (TJ SE)/TJ SE/Análise de Sistemas/Banco de Dados/2023) Considere uma tabela de um banco relacional com quatro colunas: X, Y, Z e W. Na modelagem dos dados, foram detectadas as seguintes dependências funcionais para essa tabela.

$X \rightarrow Y$

$X \rightarrow Z$

$Z \rightarrow X$

$Y \rightarrow Z$

$X \rightarrow W$

A lista das colunas que devem necessariamente ser definidas com a propriedade unique, com colunas isoladas ou em conjunto, é:

- a) X e Y, em conjunto, e W, separadamente;
- b) X, Y, Z, W, separadamente;
- c) X, Y, Z, separadamente;
- d) X, isoladamente, e ainda Y, Z, em conjunto;
- e) X, isoladamente.

Comentários:

Das Dependências Funcionais dadas, temos:

$X \rightarrow Y$

$X \rightarrow Z$

$X \rightarrow W$

Da transitividade, $X \rightarrow Y$ e $Y \rightarrow Z$, então $X \rightarrow Z$. Ou seja, $X \rightarrow \{X, Y, W, Z\}$, então X é chave.

Além disso, da transitividade, $Z \rightarrow X$ e $X \rightarrow \{X, Y, W, Z\}$, então $Z \rightarrow \{X, Y, W, Z\}$. Ou seja, Z é chave.

Finalmente, da transitividade, $Y \rightarrow Z$ e $Z \rightarrow \{X, Y, W, Z\}$, então $Y \rightarrow \{X, Y, W, Z\}$. Ou seja, Y é chave também.

Assim sendo, X, Y e Z devem ser declarados como UNIQUE.

Gabarito: C

17. (FGV/DPE RS/Banco de Dados/2023) No contexto do projeto de bancos de dados, considere uma tabela relacional com três colunas, A, B e C, cuja instância é mostrada a seguir.

A	B	C
1	100	200
2	102	200
3	97	300
4	80	400

Tomando por base apenas essa instância, analise a seguinte lista de eventuais dependências funcionais.

A $\rightarrow$ B

A $\rightarrow$ B,C

B $\rightarrow$ A,C

C $\rightarrow$ B

A,C $\rightarrow$ B

Sobre a validade dessas dependências, é correto concluir que:

- a) somente duas delas são válidas;
- b) são válidas somente as duas primeiras e a quarta;
- c) são válidas somente as três primeiras;
- d) somente quatro delas são válidas;
- e) todas são válidas.

Comentários:

A regra é simples: uma DF $X \rightarrow Y$ vale na instância se, sempre que duas linhas coincidem em X, também coincidem em Y.

$A \rightarrow B$ — Válida. Todos os valores de A são distintos (1, 2, 3, 4). Quando não há repetição no determinante, a DF é satisfeita sempre: não existe par de linhas com mesmo A para gerar contradição.

$A \rightarrow B, C$ — Válida. Mesmo raciocínio: A é todo distinto, então qualquer DF com A sozinho no lado esquerdo vale automaticamente nesta instância.

$B \rightarrow A, C$ — Válida. Todos os valores de B são distintos (100, 102, 97, 80). Pela mesma lógica, não há duas linhas com mesmo B, logo a DF é satisfeita.

$C \rightarrow B$ — Inválida. Aqui está o problema. O valor $C = 200$ aparece em duas linhas: na primeira com $B = 100$ e na segunda com $B = 102$. Como o mesmo valor de C leva a valores diferentes de B, a dependência $C \rightarrow B$ não se verifica nessa instância.

$A, C \rightarrow B$ — Válida. Os pares (A, C) são todos distintos: $(1,200)$, $(2,200)$, $(3,300)$, $(4,400)$. Sem repetição no determinante, a DF vale.

Gabarito: D

18. (FGV/TJ SE/Análise de Sistemas/2023) A normalização de esquemas de bancos de dados é feita a partir das dependências funcionais existentes entre as colunas de uma tabela. Além disso, é possível combinar dependências e derivar novas dependências a partir de axiomas preestabelecidos. Considerando uma tabela hipotética, com colunas X , Y e Z , o axioma da transitividade é definido como segue:

- a) Se $X \rightarrow Y$ e $X \rightarrow Z$ então $Z \rightarrow XY$;
- b) Se $X \rightarrow Y$ e $Y \rightarrow Z$ então $X \rightarrow Z$;
- c) Se $X \rightarrow Y$ então $XZ \rightarrow YZ$;
- d) Se $X \rightarrow Y$ então $X \rightarrow XY$;
- e) Se $XY \rightarrow Z$ então $X \rightarrow Z$.

Comentários:

A questão pede a definição do axioma da transitividade dentre os Axiomas de Armstrong.

a) Errada. Essa derivação não corresponde a nenhum axioma válido. O fato de X determinar Y e Z não cria nenhuma relação de Z determinando XY .

b) Correta. Essa é exatamente a definição da transitividade: se X determina Y , e Y determina Z , então X determina Z indiretamente.

c) Errada. Essa é a definição do axioma do aumento (ou ampliação). Podemos adicionar o mesmo atributo a ambos os lados de uma DF, e ela continua válida.

d) Errada. Essa é uma regra derivada da reflexividade, podemos passar para o lado direito o determinante. Se $X \rightarrow Y$, então $X \rightarrow XY$.

e) Errada. Essa derivação é inválida. O fato de X e Y juntos determinarem Z não significa que X sozinho determine Z .

Gabarito: B

19. (FGV/TJ SE/Administrativa/2023) A normalização de dados é o processo de analisar os esquemas de relações com base nas dependências funcionais de seus atributos e chaves primárias, envolvendo uma série de Formas Normais (FN). Observe a instância da tabela PEDIDO a seguir.

ID_PEDIDO	DATA	ID_CLIENTE	NOME_CLIENTE
1	11/07/2023	1	João
2	11/07/2023	2	Paulo

As colunas de PEDIDO têm valores atômicos e as colunas não chaves (DATA, ID_CLIENTE, NOME_CLIENTE) são totalmente dependentes da chave primária (ID_PEDIDO). Portanto, PEDIDO está normalizada nas FN:

- a) 1FN e 2FN;
- b) 1FN e 3FN;
- c) 2FN e 3FN;
- d) 2FN e 4FN;
- e) 3FN e 4FN.

Comentários:

Verificando cada Forma Normal

1FN: Valores atômicos? O próprio enunciado confirma: todas as colunas têm valores atômicos. Não há campos multivalorados ou compostos. Atende.

2FN: Dependências parciais? A 2FN exige que não existam atributos não-chave dependendo de parte da chave primária. Porém, a chave primária é ID_PEDIDO sozinho (coluna única). Quando a chave é simples (não composta), é impossível existir dependência parcial, não tem como depender de "parte" de algo que já é indivisível. Atende.

3FN — Dependências transitivas? A 3FN exige que nenhum atributo não-chave dependa de outro atributo não-chave. Aqui está o problema:

ID_PEDIDO → ID_CLIENTE (direto)

ID_CLIENTE → NOME_CLIENTE (direto)

Portanto: ID_PEDIDO → ID_CLIENTE → NOME_CLIENTE

Isso é uma dependência transitiva. O NOME_CLIENTE não depende diretamente da chave primária, ele depende do ID_CLIENTE, que por sua vez depende da chave. Em termos práticos, o nome do cliente é um fato sobre o cliente, não sobre o pedido. Não atende.

Se não está na 3FN, conseqüentemente não pode estar na 4FN.

Gabarito: A

20. (FGV/TCE TO/Tecnologia da Informação/2022) No contexto da Teoria de Projeto de bancos de dados relacionais, analise o script de criação de uma tabela.

```
CREATE TABLE T(A int not null UNIQUE,  
B int not null UNIQUE,  
C int)
```

Considere ainda as seguintes dependências funcionais acerca dos atributos A, B e C.

$A \rightarrow B$

$B \rightarrow A$

$A \rightarrow C$

$B \rightarrow C$

As dependências necessárias para que o esquema relacional acima esteja normalizado até a Forma Normal Boyce-Codd são:

- a) todas, exceto a primeira;
- b) todas, exceto a segunda;
- c) todas, exceto a terceira;
- d) todas, exceto a quarta;
- e) todas.

Comentários:

Na FNBC todo determinante das dependências funcionais deve ser chave.

Pois bem, do esquema relacional (em outras palavras, da tabela criada com "CREATE TABLE"), sabemos que tanto "A" quanto "B" são chaves devido ao comando UNIQUE, então concluímos que ela respeita as dependências funcionais: " $A \rightarrow \{A, B, C\}$ " e " $B \rightarrow \{A, B, C\}$ ".

Assim sendo, da regra da decomposição, descobrimos que a tabela respeita as dependências funcionais:

$A \rightarrow A$

$A \rightarrow B$

$A \rightarrow C$

$B \rightarrow A$

$B \rightarrow B$

$B \rightarrow C$

Como $A \rightarrow A$ e $B \rightarrow B$ são dependências funcionais triviais, podemos apenas as ignorar, então.

$A \rightarrow B$

$A \rightarrow C$

$B \rightarrow A$

$B \rightarrow C$

Assim sendo, para respeitar a FNBC, apenas essas dependências funcionais podem derivar do mundo real.

Observando as dependências funcionais do examinador a seguir, percebemos que não há nenhuma em excesso, visto que batem exatamente com as que deduzimos. Então, gabarito letra E, já que "todas" são necessárias.

$A \rightarrow B$

$B \rightarrow A$

$A \rightarrow C$

$B \rightarrow C$

Gabarito: E

21. (FGV/Ana (MPE SC)/MPE SC/Dados e Pesquisas/2022) A modelagem de bancos de dados passa pela análise das relações e comportamento dos dados que futuramente constituirão o conteúdo desses bancos. Para bancos de dados relacionais, essa modelagem passa pelo

levantamento das dependências funcionais que eventualmente possam ser depreendidas em cada caso.

Como um exemplo, considere um banco de dados que armazena a data de nascimento, o CPF (Cadastro de Pessoas Físicas) e a CNH (Carteira Nacional de Habilitação) de um grupo de pessoas, no qual todas possuem CPF e CNH.

Dado que no Brasil o CPF e a CNH são individualizados, as dependências funcionais que devem ser consideradas, em conjunto, são:

a) CPF → DataNascimento

CNH → DataNascimento

CPF → CNH

CNH → CPF

b) CPF → DataNascimento

DataNascimento → CNH

CNH → CPF

c) CPF → DataNascimento

DataNascimento → CPF

CPF → CNH

d) CPF → CNH

CNH → CPF

e) CPF → DataNascimento

CNH → DataNascimento

Comentários:

a) Correta. Como CPF e CNH são individualizados (cada pessoa tem um único CPF e uma única CNH), sabendo o CPF conseguimos determinar a data de nascimento e a CNH daquela pessoa, e sabendo a CNH também conseguimos determinar a data de nascimento e o CPF. Essas são verdades do mundo real que se traduzem em dependências funcionais válidas: CPF determina DataNascimento, CNH determina DataNascimento, CPF determina CNH e CNH determina CPF.

b) Errada. Inclui DataNascimento determinando CNH, o que não faz sentido, pois várias pessoas podem ter a mesma data de nascimento e CNHs diferentes. Dependências funcionais refletem regras do mundo real, e data de nascimento não individualiza ninguém.

c) Errada. Inclui DataNascimento determinando CPF, o que é falso, pois muitas pessoas nascem no mesmo dia e possuem CPFs diferentes. Além disso, falta a dependência CNH determinando DataNascimento e CNH determinando CPF, que são válidas já que a CNH é individualizada.

d) Errada. Embora CPF determinar CNH e CNH determinar CPF estejam corretas, faltam as dependências envolvendo DataNascimento. Tanto o CPF quanto a CNH determinam a data de nascimento da pessoa, e essas dependências precisam ser consideradas.

e) Errada. Embora CPF determinar DataNascimento e CNH determinar DataNascimento estejam corretas, faltam as dependências CPF determinar CNH e CNH determinar CPF. Como ambos são individualizados, sabendo um conseguimos determinar o outro, e essas dependências precisam fazer parte do conjunto.

Gabarito: A

22. (FGV/AUFC (TCU)/TCU/Controle Externo/Auditoria Governamental/2022) Para a questão a seguir, considere uma tabela relacional R , com atributos W, X, Y, Z , e o conjunto de dependências funcionais identificadas para esses atributos.

$X \rightarrow Y$

$X \rightarrow Z$

$Z \rightarrow X$

$Z \rightarrow W$

Analise o conjunto de dependências funcionais inferidas abaixo a partir do conjunto de atributos e dependências funcionais presentes na tabela R , como descrita anteriormente.

(1) $X \rightarrow Y Z W$

(2) $X \rightarrow W$

(3) $X W \rightarrow Y W$

(4) $X Y Z W \rightarrow X Y$

(5) $Y \rightarrow Z$

À luz dos axiomas da teoria de projeto de bancos de dados aplicáveis nesse caso, é correto concluir que, dentre essas dependências inferidas:

a) somente 2 é válida;

b) somente 1, 2 e 5 são válidas;

c) somente 1, 2, 3 e 4 são válidas;

d) somente 2, 3, 4 e 5 são válidas;

e) todas são válidas.

Comentários:

(1) $X \rightarrow Y Z W$

Sabemos que $X \rightarrow Y$ e $X \rightarrow Z$. Da transitividade, como $X \rightarrow Z$ e $Z \rightarrow W$, então $X \rightarrow W$. Ou seja, é correto falar que X determina Y, Z e W : $X \rightarrow Y Z W$

(2) $X \rightarrow W$

Da transitividade, como $X \rightarrow Z$ e $Z \rightarrow W$, então $X \rightarrow W$.

(3) $X W \rightarrow Y W$

Como $X \rightarrow Y$ foi dada, podemos aumentá-la com W nos dois lados: $XW \rightarrow YW$.

(4) $X Y Z W \rightarrow X Y$

Válida por Reflexividade. XY está contido em $XYZW$, e um conjunto sempre determina seus subconjuntos.

(5) $Y \rightarrow Z$

Nenhuma dependência dada parte de Y , então é impossível chegar nessa dependência.

Gabarito: C

23. (FGV/AUFC (TCU)/TCU/Controle Externo/Auditoria Governamental/2022) Para a questão

a seguir, considere uma tabela relacional R, com atributos W, X, Y, Z, e o conjunto de dependências funcionais identificadas para esses atributos.

$X \rightarrow Y$

$X \rightarrow Z$

$Z \rightarrow X$

$Z \rightarrow W$

Com referência à tabela R, definida anteriormente, considere o esboço de um comando SQL para a criação da tabela.

```
create table R (
```

```
  X int not null,
```

```
  Y int not null,
```

```
  W int not null,
```

```
  Z int not null,
```

```
  constraint ...
```

```
)
```

De acordo com as dependências funcionais de R, e com a Forma Normal de Boyce-Codd, a definição correta das chaves (por meio de constraints) aplicáveis e necessárias para essa tabela deveria ser:

a)

```
constraint c1 unique (X)
```

```
constraint c2 unique (Z)
```

b)

```
constraint c1 unique (X, Z)
```

c)

```
constraint c1 unique (X, Y, W)
```

d)

```
constraint c1 unique (X, Y, Z)
```

e)

```
constraint c1 unique (X)
```

```
constraint c2 unique (Y)
```

```
constraint c3 unique (Z)
```

Comentários:

Para estar na forma normal de Boyce Codd, apenas chaves devem ser determinantes. Então, precisamos descobrir os determinantes para definirmos depois as chaves usando UNIQUE.

Dependências Funcionais dadas:

$X \rightarrow Y$

$X \rightarrow Z$

Da transitividade, temos que $X \rightarrow Z$ e $Z \rightarrow W$, ou seja, $X \rightarrow \{X, Y, W, Z\}$. Assim, X é chave.

Além disso, da transitividade Da transitividade, temos que $Z \rightarrow X$ e $X \rightarrow \{X, Y, W, Z\}$, ou seja, $Z \rightarrow \{X, Y, W, Z\}$. Assim, Z também é chave.

Gabarito: A

24. (FGV/Prof (SEAD AP)/SEAD AP/Educação Básica Profissional/Informática/2022) No projeto de bancos de dados relacionais, a identificação das dependências funcionais é fundamental no processo de

- a) dimensionamento físico do banco de dados.
- b) estabelecimento dos privilégios de acesso.
- c) identificação das chaves estrangeiras.
- d) normalização das tabelas.
- e) otimização das consultas.

Comentários:

a) Errada. Dimensionamento físico trata de espaço em disco e infraestrutura, não tem relação direta com dependências funcionais.

b) Errada. Privilégios de acesso dizem respeito a permissões de usuários, não a dependências funcionais.

c) Errada. Chaves estrangeiras são restrições de integridade entre tabelas, mas não são definidas a partir da identificação de dependências funcionais.

d) Correta. As dependências funcionais são o conceito central usado para definir várias formas normais (a 2FN, a 3FN e a Forma Normal de Boyce Codd). Sem identificar as dependências funcionais, não é possível avaliar se uma tabela respeita essas formas normais, tornando-as fundamentais no processo de normalização das tabelas.

e) Errada. Otimização de consultas envolve melhorar o desempenho das buscas no banco, não se baseia na identificação de dependências funcionais.

Gabarito: D

25. (FGV/Ana MP (MPE GO)/MPE GO/Analista em Informática/2022) No Campeonato Brasileiro de Futebol, todos os times jogam entre si duas vezes, alternando os papéis de mandante e visitante. Um time não pode jogar mais de uma vez na mesma data.

Nesse contexto, considere uma tabela de bancos de dados que armazene simplificada essas informações, com as seguintes colunas:

- time_mandante
- time_visitante
- data
- gols_mandante
- gols_visitante

À luz dessas premissas, assinale a lista com todas as combinações de colunas que devem, necessária e exclusivamente, ser estabelecidas como restrições (primary key ou unique) para atender à Forma Normal Boyce-Codd.

- a) (time_mandante, time_visitante)
- b) (time_mandante, data)
- (time_visitante, data)

- c) (time_mandante, data, gols_mandante)
- (time_visitante, data, gols_visitante)
- d) (time_mandante, time_visitante)
- (time_visitante, time_mandante)
- e) (time_mandante, time_visitante)
- (time_mandante, data)
- (time_visitante, data)

Comentários:

a) Errada. Apenas (time_mandante, time_visitante) identifica o jogo entre dois times, mas não cobre a regra de que um time não pode jogar mais de uma vez na mesma data. Sobraria a situação em que a "data" é determinada por algo que não é chave, violando a FNBC.

b) Errada. Essas duas combinações garantem que cada time joga uma vez por data, mas falta a combinação (time_mandante, time_visitante), que também é uma chave candidata, pois cada par de times só se enfrenta uma vez como mandante e visitante. Sem declarar essa restrição, a FNBC não estaria satisfeita, já que essa combinação também determina os demais atributos.

c) Errada. Incluir gols_mandante e gols_visitante nas chaves não faz sentido, pois esses são atributos comuns (resultados do jogo), não determinantes de outros atributos. Chaves devem ser o mínimo necessário para identificar univocamente cada registro.

d) Errada. (time_mandante, time_visitante) e (time_visitante, time_mandante) representam essencialmente a mesma informação invertida, e não cobrem a regra de que um time só joga uma vez por data. A "data" continuaria sendo determinada por algo que não foi declarado como chave, violando a FNBC.

e) Correta. A FNBC exige que apenas chaves possam determinar outros atributos. Pelas regras do campeonato, existem três chaves candidatas: (time_mandante, time_visitante) identifica o confronto único nessa condição; (time_mandante, data) identifica o jogo porque um time mandante só joga uma vez por data; e (time_visitante, data) pela mesma razão. Cada uma dessas combinações determina funcionalmente todos os outros atributos da tabela. Se qualquer uma delas não fosse declarada como restrição, teríamos um determinante que não é chave, violando a FNBC. Por isso as três combinações são necessárias.

Gabarito: E

26. (FGV/Ag TE (SEFAZ BA)/SEFAZ BA/Administração e Finanças/2022) Leia o fragmento a seguir.

"Uma tabela está na _____ se, e somente se, para cada _____, onde X e A são atributos simples ou compostos, uma das duas condições precisam ser mantidas: ou o atributo X é uma _____, ou o atributo A é uma chave candidata. Se o atributo A é membro de uma chave candidata".

Assinale a opção cujos itens completam corretamente as lacunas do fragmento acima.

- a) forma normal boyce-codd – dependência multivalorada – chave primária.
- b) forma normal boyce-codd – dependência funcional não trivial $X \rightarrow A$ – chave primária.
- c) terceira forma normal – dependência funcional trivial $X \rightarrow A$ – chave candidata.

d) terceira forma normal – dependência funcional não trivial $X \rightarrow A$ – superchave.

e) quarta forma normal – dependência funcional trivial $X \rightarrow A$ – chave candidata.

Comentários:

a) **Errada.** A Forma Normal de Boyce-Codd exige que apenas chaves possam determinar outros atributos, sem abrir exceção para atributos que sejam parte de chave candidata. Além disso, dependência multivalorada é assunto da 4FN, não da BCNF.

b) **Errada.** A BCNF não abre a exceção mencionada no fragmento (de que A pode ser membro de chave candidata). Além disso, a notação com seta dupla ($X \twoheadrightarrow A$) se refere a dependência multivalorada, não funcional.

c) **Errada.** A notação com seta dupla ($X \twoheadrightarrow A$) se refere a dependência multivalorada, não a dependência funcional. Além disso, o termo "trivial" não se encaixa no contexto descrito.

d) **Correta.** A 3FN proíbe que atributos comuns (não chave) determinem outros atributos comuns. Porém, ela deixa uma brecha: permite que um atributo comum determine um atributo que faça parte de uma chave candidata. O fragmento descreve exatamente isso: para cada dependência funcional não trivial X determina A, ou X é uma superchave, ou A (determinado) é membro de uma chave candidata. Essa segunda condição é justamente a brecha que diferencia a 3FN da Forma Normal de Boyce-Codd.

e) **Errada.** A 4FN trata de dependências multivaloradas, não de dependências funcionais. Além disso, o termo "trivial" e a notação com seta dupla não se encaixam no contexto do fragmento.

Gabarito: D

27. (FGV/ES (SEMSA Manaus)/Pref Manaus/Analista de Banco de Dados/2022) Suponha a existência dos atributos V, W, X, Y, Z numa tabela de banco de dados relacional, e considere as seguintes assertivas sobre dependências funcionais e seus axiomas.

I. Se $W \rightarrow X$ e $W \rightarrow Y$ então $X \rightarrow Y$

II. Se $X \rightarrow Y$ então $XW \rightarrow YW$

III. $XY \rightarrow X$

IV. Se $X \rightarrow Y$ e $Y \rightarrow Z$ e $Z \rightarrow V$ então $X \rightarrow V$

Estão corretas as afirmativas

a) I, II, III e IV.

b) I e III, somente.

c) I, II e IV, somente.

d) II e IV, somente.

e) II, III e IV, somente.

Comentários:

A afirmativa I diz que se W determina X e W determina Y, então X determina Y. Isso está errado, pois o fato de W determinar tanto X quanto Y não significa que X determine Y. Não existe nenhuma regra que permita essa conclusão.

A afirmativa II diz que se X determina Y, então XW determina YW. Isso está correto e corresponde à regra da augmentação, que permite adicionar um atributo extra (no caso, W) aos dois lados de uma dependência funcional sem invalidá-la.

A afirmativa III diz que XY determina X. Isso está correto e corresponde à regra da reflexividade, que diz que um conjunto de atributos sempre determina suas partes. O conjunto formado por X e Y naturalmente determina X, que é uma parte dele.

A afirmativa IV diz que se X determina Y, Y determina Z e Z determina V, então X determina V. Isso está correto e corresponde à regra da transitividade aplicada duas vezes: primeiro encadeamos X determina Y com Y determina Z para concluir que X determina Z, e depois encadeamos X determina Z com Z determina V para concluir que X determina V.

Gabarito: E

28. (FGV/AS (SEMSA Manaus)/Pref Manaus/Programador de Computador/2022) Em todos os comandos, supõe-se que NULL é tratado como um valor desconhecido (unkown).

T1		T2		T3		
A	B	C	D	A	C	E
1	3	100	42	1	100	12
2	2	110	55	1	110	12
4	2	120	33	2	100	NULL

Tomando por base apenas as instâncias das tabelas do banco de dados EXEMPLO, anteriormente especificado, é possível confirmar a existência da dependência funcional

- a) $A \rightarrow B$
- b) $A \rightarrow C$
- c) $B \rightarrow A$
- d) $C \rightarrow A$
- e) $E \rightarrow C$

Comentários:

a) **Correta.** A dependência A seta B indica que o atributo A determina o atributo B. Isso significa que, para cada valor de A, existe sempre um único valor de B associado. Ao analisar os dados da tabela, não há dois registros com o mesmo valor de A e valores diferentes de B, confirmando essa dependência funcional.

b) **Errada.** A seta C não se confirma porque, ao observar os dados, existem registros com o mesmo valor de A associados a valores diferentes de C, o que viola a definição de dependência funcional.

c) **Errada.** B seta A não se confirma porque há registros com o mesmo valor de B associados a valores diferentes de A, quebrando a regra de que o determinante define um único valor no dependente.

d) **Errada.** C seta A não se confirma porque há valores repetidos de C que aparecem com valores diferentes de A nos registros da tabela.

e) **Errada.** E seta C não se confirma porque há registros com o mesmo valor de E associados a valores diferentes de C.

Gabarito: A

29. (FGV/Ana (FunSaúde CE)/FunSaúde CE/TI e Infraestrutura de Informática/2021) Considere a definição e as instâncias das tabelas de bancos de dados CLUBE e JOGO exibidas a seguir.

nome
Barcelona
Boca Juniors
The Strongest

JOGO

mandante	visitante	golsM	golsV
Barcelona	Boca Juniors	1	0
Barcelona	The Strongest	NULL	NULL
Boca Juniors	Barcelona	0	0
Boca Juniors	The Strongest	3	0
The Strongest	Barcelona	2	0
The Strongest	Boca Juniors	2	0

Cada clube deve jogar quatro vezes, duas como mandante e duas como visitante.

As colunas golsM e golsV registram o número de gols dos times mandantes e visitantes, respectivamente, em cada jogo. Ambas são nulas enquanto o jogo não for realizado.

Dependências funcionais têm um papel importante na teoria de bancos de dados. Tomando por base a instância da tabela JOGO, definida anteriormente, assinale a dependência funcional que pode ser corretamente depreendida.

a) mandate $\rightarrow$ golsM

b) mandate, visitante $\rightarrow$ golsM

c) golsM $\rightarrow$ mandante

d) golsM, golsV $\rightarrow$ mandante, visitante

e) visitante $\rightarrow$ gols, golsV

Comentários:

a) **Errada.** Saber apenas o mandante não é suficiente para determinar golsM, porque um mesmo clube pode ser mandante em mais de um jogo (Barcelona é mandante duas vezes, por exemplo), então o valor de golsM pode variar entre esses jogos.

b) **Correta.** Sabendo quem é o mandante e quem é o visitante, identificamos um jogo específico (cada combinação mandante + visitante acontece uma única vez), e assim conseguimos determinar o valor de golsM. Isso é uma dependência funcional válida: mandante, visitante determina golsM.

c) **Errada.** O número de gols do mandante não determina quem é o mandante. Vários jogos diferentes podem ter o mesmo placar de gols para o mandante, então golsM sozinho não identifica qual clube é o mandante.

d) **Errada.** Saber os gols do mandante e do visitante não é suficiente para determinar quem jogou. Jogos diferentes podem terminar com o mesmo placar, então golsM e golsV juntos não identificam necessariamente mandante e visitante.

e) **Errada.** Saber apenas o visitante não determina gols nem golsV, pois o mesmo clube pode ser visitante em mais de um jogo, com resultados diferentes em cada um.

Gabarito: B

30. (FGV/Ana TI (BANESTES)/BANESTES/Desenvolvimento de Sistemas/2021) Considere um banco de dados que contém uma tabela com os atributos CPF, Nome, RG e Endereço, com milhões de registros. Sabe-se que cada pessoa tem seus próprios CPF e RG e um único nome, embora possa haver homônimos.

Sendo assim, a dependência funcional que NÃO existe entre os atributos dessa tabela é:

a) CPF → Nome

b) CPF → RG

c) Endereço → CPF

d) RG → Endereço

e) RG → Nome

Comentários:

a) **Errada.** Como cada pessoa tem seu próprio CPF e um único nome, sabendo o CPF sabemos o nome da pessoa. Essa dependência funcional existe.

b) **Errada.** Como cada pessoa tem seus próprios CPF e RG, sabendo o CPF sabemos qual é o RG daquela pessoa. Essa dependência funcional existe.

c) **Correta.** Um mesmo endereço pode abrigar várias pessoas diferentes, cada uma com seu próprio CPF. Ou seja, saber o endereço não determina um único CPF. Portanto, essa dependência funcional não existe, sendo a resposta da questão.

d) **Errada.** Como cada pessoa tem seu próprio RG e um único endereço, sabendo o RG identificamos a pessoa e consequentemente seu endereço. Essa dependência funcional existe.

e) **Errada.** Como cada pessoa tem seu próprio RG e um único nome, sabendo o RG identificamos a pessoa e consequentemente seu nome. Essa dependência funcional existe.

Gabarito: C

31. (FGV/ATCE (TCE,AM)/TCE AM/Auditoria de Tecnologia da Informação/2021) Na questão, considere uma tabela de banco de dados T cuja instância é exibida a seguir.

A	B	C	D
12	2	3	1
14	3	8	2
18	2	9	3
21	5	4	4

Supondo-se que a tabela T (descrita anteriormente) esteja normalizada até a forma normal Boyce-Codd, a dependência funcional que NÃO se verifica nessa tabela é:

- a) $A \rightarrow B$
- b) $B \rightarrow D$
- c) $C \rightarrow B$
- d) $C \rightarrow D$
- e) $A, D \rightarrow B$

Comentários:

a) Errada. Olhando a tabela, cada valor de A aparece apenas uma vez (12, 14, 18, 21), então para cada valor de A existe um único valor de B associado (não há problema em B ser repetido, o importante é o cada único determinante A determinar um único valor de B sempre igual). Essa dependência se verifica.

b) Correta. Observe que o valor $B=2$ aparece duas vezes: na primeira linha com $D=1$ e na terceira linha com $D=3$. Ou seja, o mesmo valor de B está associado a valores diferentes de D. Isso viola a definição de dependência funcional, pois se B determinasse D, jamais poderiam existir dois registros com o mesmo B e D diferentes. Portanto, B não determina D.

c) Errada. Cada valor de C (3, 8, 9, 4) aparece apenas uma vez na tabela, então para cada valor de C existe um único valor de B. Essa dependência se verifica.

d) Errada. Cada valor de C aparece apenas uma vez, então para cada valor de C existe um único valor de D. Essa dependência se verifica.

e) Errada. Cada combinação de A e D é única na tabela, então para cada par (A, D) existe um único valor de B. Essa dependência se verifica.

Gabarito: B

LISTA DE QUESTÕES - FGV



1. (FGV/AMAZUL/Técnico de Informática/2026) No processo de projeto lógico de bancos de dados relacionais, a normalização organiza os atributos das relações de forma a eliminar redundâncias indesejadas e anomalias de atualização.

Dentro desse encadeamento, a forma normal que tem sua definição fundamentada direta e exclusivamente no conceito de dependência funcional total é a

- a) primeira forma normal (1FN).
- b) segunda forma normal (2FN).
- c) terceira forma normal (3FN).
- d) forma normal Boyce-Codd (FNBC).
- e) quarta forma normal (4FN).

2. (FGV/AAD (TCE,RR)/TCE RR/Tecnologia da Informação/Banco de Dados/2025) O conceito de dependência funcional é fundamental na análise de esquemas relacionais, sendo uma ferramenta básica para medir a adequação dos agrupamentos de atributos. Dependências funcionais podem ser utilizadas para descrever melhor um esquema de relação ao especificar restrições que devem ser mantidas entre seus atributos.

As dependências funcionais problemáticas podem ser eliminadas por meio do processo de normalização, buscando alcançar as várias formas normais.

As formas normais que buscam eliminar novos conceitos de dependência funcional que são extensões e especializações do conceito original são

- a) 1FN e 5FN.
- b) 2FN e 3FN.
- c) 3FN e Forma Normal de Boyce-Codd (FNBC).
- d) 4FN e 5FN.
- e) 3FN e 4FN.

3. (FGV/AAD (TCE,RR)/TCE RR/Tecnologia da Informação/Banco de Dados/2025) Considerando o processo de normalização de um banco de dados relacional, avalie as afirmativas a seguir.

I. Um dos objetivos da normalização é minimizar redundância.

II. A normalização oferece uma estrutura formal para analisar esquemas de relação com base em suas chaves.

III. Desnormalização é o processo de armazenar a junção de relações na forma normal mais baixa como uma relação básica.

Está correto o que se afirma em

a) I, apenas.

b) II, apenas.

c) I e II, apenas.

d) I e III, apenas.

e) II e III, apenas.

4. (FGV/TJ TRT24/TRT 24/Apoio Especializado/Tecnologia da Informação/2025) Durante a instalação de um SGBD, o administrador de dados decidiu aplicar princípios de normalização. Assinale a opção que indica a finalidade de adotar tais princípios.

a) Automatizar a criação de índices em todas as colunas, aumentando a velocidade das consultas.

b) Reduzir redundâncias, otimizando a manutenção do banco de dados.

c) Garantir alocação dinâmica de memória no servidor, acelerando o acesso aos dados.

d) Otimizar a estrutura, reduzindo a frequência de backups incrementais.

e) Armazenar todas as chaves estrangeiras em uma única tabela, simplificando as relações entre as tabelas.

5. (FGV/AJ (TJ AP)/TJ AP/Apoio Especializado/Tecnologia da Informação Banco de Dados/2024) Considere uma tabela relacional com atributos (colunas) A, B, C, D e E, na qual as dependências funcionais conhecidas são:

A -> B

A -> C

A -> D

D -> E

O comando de criação dessa tabela é mostrado a seguir.

```
create table X(A int, B int, C int, D int, E int,  
constraint K1 unique (A),  
constraint K2 unique (D)  
)
```

De acordo com as formas normais Primeira, Segunda, Terceira e Boyce-Codd, a dependência funcional adicional necessária e suficiente para a validade desse esquema é:

- a) A -> E;
- b) A, D -> E;
- c) D -> A;
- d) D -> B, C, D;
- e) E -> A.

6. (FGV/Tec NS (TJ MS)/TJ MS/Analista de Sistemas Computacionais/Analista de Banco de Dados/2024) A normalização de dados é o processo de analisar os esquemas de relações com base nas dependências funcionais de seus atributos e chaves primárias, envolvendo uma série de Formas Normais (FN).

Observe o esquema da tabela "Andamento_Processo", a seguir.

IDAND	IDPRC	DATA	DESCRICAO	ADV_RESPONSAVEL
1	1	2024-01-11	Petição Inicial	Maria
2	1	2024-01-25	Audiência	Ana
3	2	2024-01-30	Interrogatório	Luiza

A tabela "Andamento_Processo" possui chave primária composta, atributos com valores atômicos e ausências de dependências transitivas entre os atributos não chave.

Com base nisso, é correto afirmar que "Andamento_Processo" está normalizada na FN:

- a) 1FN;
- b) 2FN;
- c) 3FN;
- d) 4FN;
- e) 5FN.

7. (FGV/Tec NS (TJ MS)/TJ MS/Analista de Sistemas Computacionais/Analista de Sistemas/2024) A normalização de dados é o processo de analisar os esquemas de relações com base nas dependências funcionais de seus atributos e chaves primárias, envolvendo uma série de Formas Normais (FN). Observe o esquema da tabela "Pedido", a seguir.

ID_PEDIDO	ID_CLIENTE	DATA	NOME_CLIENTE
1	2	16/04/2024	Luiza
2	3	16/04/2024	Júlia

A tabela "Pedido" possui chave primária composta (ID_PEDIDO, ID_CLIENTE), atributos com valores atômicos e dependência parcial de coluna não chave (NOME_CLIENTE) com parte da chave primária (ID_CLIENTE).

Com base nisso, "Pedido" está normalizada na FN:

- a) 1FN;
- b) 2FN;
- c) 3FN;
- d) 4FN;
- e) 5FN.

8. (FGV/Prof NS (SES MT)/SES MT/Analista de Sistemas/Administrador de Banco de Dados ou Infraestrutura e O&M/2024) Existem diversas formas normais que podem ser utilizadas na normalização de esquemas relacionais. A forma normal Boyce- Codd (FNBC) foi proposta para ser uma simplificação, mas descobriu-se que ela é mais rigorosa que a forma normal que ela pretendia simplificar.

A forma normal que a FNBC pretendia simplificar é a

- a) 1FN.

- b) 2FN.
- c) 3FN.
- d) 4FN.

9. (FGV/STN/Tecnologia da Informação/Operação e Infraestrutura/2024) Com referência à normalização de bancos de dados, considere uma tabela relacional T, com colunas C, H, N, P, na qual as dependências funcionais identificadas são exibidas a seguir.

C → H

C → P

C → N

P → C

O comando SQL utilizado na criação dessa tabela é exibido a seguir.

```
create table T ( C int not null unique,
```

```
H int null unique,
```

```
P int null unique,
```

```
N int not null
```

```
)
```

Assinale a dependência funcional adicional necessária para que o esquema acima esteja de acordo com a forma normal Boyce-Codd.

- a) C, H, P → N
- b) H → P
- c) N → P
- d) N → H
- e) P, H → C

10. (FGV/TJ (TRF 1ª Região)/TRF 1/Apoio Especializado/Tecnologia da Informação/2024) Roberta, funcionária da ORG_A, está utilizando instruções em SQL para fazer inclusões, alterações e exclusões de registros em um banco de dados que contém as informações de funcionários que participaram de determinados treinamentos promovidos pela ORG_A. Roberta notou algumas anomalias de exclusão, pois, ao comandar a instrução SQL para excluir um funcionário desligado da organização, alguns treinamentos desapareceram.

Considerando que as instruções em SQL executadas por Roberta estão sintática e semanticamente corretas, a modelagem do banco de dados possui problemas relacionados à existência de:

- a) dependências multivaloradas de junção;
- b) chave primária com mais de um campo não atômico;
- c) atributos com dependências não funcionais das chaves candidatas;
- d) dependências funcionais entre as chaves candidatas e a chave primária;
- e) dependências transitivas de atributos não chaves em relação à chave primária.

11. (FGV/AJ (TRF 1ª Região)/TRF 1/Apoio Especializado/Análise de Dados/2024) Na teoria do processo de normalização em bancos de dados relacionais, o conceito de chave é definido por “um conjunto de uma ou mais colunas, tal que não possa haver duas linhas, t_1 , t_2 , cujos valores presentes nessas colunas sejam idênticos em t_1 e t_2 ”.

Nesse contexto, considere uma tabela T, com colunas A, B, C e D, para as quais são válidas as seguintes dependências funcionais:

A $\rightarrow$ B

B $\rightarrow$ C

B $\rightarrow$ D

C $\rightarrow$ B

Supondo que, adicionalmente, a dependência funcional

C $\rightarrow$ A

seja válida, é correto concluir que, obrigatoriamente, a tabela T deve ser:

- a) declarada com duas chaves distintas;
- b) declarada com três chaves distintas;
- c) declarada com uma chave apenas;
- d) particionada em duas tabelas;
- e) particionada em três tabelas.

12. (FGV/FTE (SEFAZ MT)/SEFAZ MT/2023) A normalização das estruturas de dados é uma etapa importante do processo de modelagem relacional para eliminar distorções ou anomalias no modelo.

Diz-se que uma tabela está na segunda forma normal, de acordo com as regras de normalização, se ela está na primeira forma normal e

- a) as linhas da tabela são unívocas, sem chaves compostas e todos os seus atributos são atômicos.
- b) as linhas não contêm itens repetitivos, atributos com valores nulos e nem multidimensionais.
- c) cada uma das colunas não pertencentes à chave primária não for dependente parcialmente dessa chave.
- d) contém apenas chaves externas e conecta-se a outras tabelas por múltiplas junções.
- e) as colunas não pertencentes à chave primária ficam determinadas transitivamente por esta.

13. (FGV/DPE RS/Apoio Especializado (TI)/Banco de Dados/2023) A primeira forma normal, utilizada no processo de normalização de um banco de dados relacional, estabelece que:

- a) não haja linhas duplicadas numa tabela;
- b) não haja colunas idênticas numa tabela;
- c) em nenhuma das linhas de uma tabela haja valores múltiplos armazenados na mesma coluna;
- d) em qualquer tabela deve haver uma coluna que possa ser declarada como chave primária;
- e) não haja transitividade de dependências funcionais entre as colunas.

14. (FGV/Pref Niterói/Tecnologia da Informação/2023) Dependências funcionais são usadas no processo de normalização de tabelas em projetos de bancos de dados relacionais e possuem propriedades que permitem a derivação de novas dependências. Considerando-se os atributos A, B, C de uma tabela relacional T, a derivação que NÃO é válida é:

- a) $A, B, C \rightarrow A, B$;
- b) se $A \rightarrow B$ e $A \rightarrow C$ então $A \rightarrow B, C$;
- c) se $A \rightarrow B$ e $B \rightarrow C$ então $A \rightarrow C$;
- d) se $A \rightarrow C$ então $A, B \rightarrow B, C$;
- e) se $B \rightarrow A$ e $C \rightarrow A$ então $B \rightarrow C$.

15. (FGV/TJ (TJ SE)/TJ SE/Administrativa/Programação de Sistemas/2023) O processo de normalização de dados consiste em projetar relações normalizadas para organizar os dados em um banco de dados relacional e aumentar a:

- a) anomalia de inserção;
- b) anomalia de exclusão;

- c) integridade de dados;
- d) redundância de dados;
- e) anomalia de atualização.

16. (FGV/AJ (TJ SE)/TJ SE/Análise de Sistemas/Banco de Dados/2023) Considere uma tabela de um banco relacional com quatro colunas: X, Y, Z e W. Na modelagem dos dados, foram detectadas as seguintes dependências funcionais para essa tabela.

$X \rightarrow Y$

$X \rightarrow Z$

$Z \rightarrow X$

$Y \rightarrow Z$

$X \rightarrow W$

A lista das colunas que devem necessariamente ser definidas com a propriedade unique, com colunas isoladas ou em conjunto, é:

- a) X e Y, em conjunto, e W, separadamente;
- b) X, Y, Z, W, separadamente;
- c) X, Y, Z, separadamente;
- d) X, isoladamente, e ainda Y, Z, em conjunto;
- e) X, isoladamente.

17. (FGV/DPE RS/Banco de Dados/2023) No contexto do projeto de bancos de dados, considere uma tabela relacional com três colunas, A, B e C, cuja instância é mostrada a seguir.

A	B	C
1	100	200
2	102	200
3	97	300
4	80	400

Tomando por base apenas essa instância, analise a seguinte lista de eventuais dependências funcionais.

A -> B

$A \rightarrow B, C$

$B \rightarrow A, C$

$C \rightarrow B$

$A, C \rightarrow B$

Sobre a validade dessas dependências, é correto concluir que:

- a) somente duas delas são válidas;
- b) são válidas somente as duas primeiras e a quarta;
- c) são válidas somente as três primeiras;
- d) somente quatro delas são válidas;
- e) todas são válidas.

18. (FGV/TJ SE/Análise de Sistemas/2023) A normalização de esquemas de bancos de dados é feita a partir das dependências funcionais existentes entre as colunas de uma tabela. Além disso, é possível combinar dependências e derivar novas dependências a partir de axiomas preestabelecidos. Considerando uma tabela hipotética, com colunas X, Y e Z, o axioma da transitividade é definido como segue:

- a) Se $X \rightarrow Y$ e $X \rightarrow Z$ então $Z \rightarrow XY$;
- b) Se $X \rightarrow Y$ e $Y \rightarrow Z$ então $X \rightarrow Z$;
- c) Se $X \rightarrow Y$ então $XZ \rightarrow YZ$;
- d) Se $X \rightarrow Y$ então $X \rightarrow XY$;
- e) Se $XY \rightarrow Z$ então $X \rightarrow Z$.

19. (FGV/TJ SE/Administrativa/2023) A normalização de dados é o processo de analisar os esquemas de relações com base nas dependências funcionais de seus atributos e chaves primárias, envolvendo uma série de Formas Normais (FN). Observe a instância da tabela PEDIDO a seguir.

ID_PEDIDO	DATA	ID_CLIENTE	NOME_CLIENTE
1	11/07/2023	1	João
2	11/07/2023	2	Paulo

As colunas de PEDIDO têm valores atômicos e as colunas não chaves (DATA, ID_CLIENTE, NOME_CLIENTE) são totalmente dependentes da chave primária (ID_PEDIDO). Portanto, PEDIDO está normalizada nas FN:

- a) 1FN e 2FN;
- b) 1FN e 3FN;
- c) 2FN e 3FN;
- d) 2FN e 4FN;
- e) 3FN e 4FN.

20. (FGV/TCE TO/Tecnologia da Informação/2022) No contexto da Teoria de Projeto de bancos de dados relacionais, analise o script de criação de uma tabela.

```
CREATE TABLE T(A int not null UNIQUE,  
B int not null UNIQUE,  
C int)
```

Considere ainda as seguintes dependências funcionais acerca dos atributos A, B e C.

- $A \rightarrow B$
- $B \rightarrow A$
- $A \rightarrow C$
- $B \rightarrow C$

As dependências necessárias para que o esquema relacional acima esteja normalizado até a Forma Normal Boyce-Codd são:

- a) todas, exceto a primeira;
- b) todas, exceto a segunda;
- c) todas, exceto a terceira;
- d) todas, exceto a quarta;
- e) todas.

21. (FGV/Ana (MPE SC)/MPE SC/Dados e Pesquisas/2022) A modelagem de bancos de dados passa pela análise das relações e comportamento dos dados que futuramente constituirão o conteúdo desses bancos. Para bancos de dados relacionais, essa modelagem passa pelo levantamento das dependências funcionais que eventualmente possam ser depreendidas em cada caso.

Como um exemplo, considere um banco de dados que armazena a data de nascimento, o CPF

(Cadastro de Pessoas Físicas) e a CNH (Carteira Nacional de Habilitação) de um grupo de pessoas, no qual todas possuem CPF e CNH.

Dado que no Brasil o CPF e a CNH são individualizados, as dependências funcionais que devem ser consideradas, em conjunto, são:

a) CPF $\rightarrow$ DataNascimento

CNH $\rightarrow$ DataNascimento

CPF $\rightarrow$ CNH

CNH $\rightarrow$ CPF

b) CPF $\rightarrow$ DataNascimento

DataNascimento $\rightarrow$ CNH

CNH $\rightarrow$ CPF

c) CPF $\rightarrow$ DataNascimento

DataNascimento $\rightarrow$ CPF

CPF $\rightarrow$ CNH

d) CPF $\rightarrow$ CNH

CNH $\rightarrow$ CPF

e) CPF $\rightarrow$ DataNascimento

CNH $\rightarrow$ DataNascimento

22. (FGV/AUFC (TCU)/TCU/Controle Externo/Auditoria Governamental/2022) Para a questão a seguir, considere uma tabela relacional R, com atributos W, X, Y, Z, e o conjunto de dependências funcionais identificadas para esses atributos.

$X \rightarrow Y$

$X \rightarrow Z$

$Z \rightarrow X$

$Z \rightarrow W$

Analise o conjunto de dependências funcionais inferidas abaixo a partir do conjunto de atributos e dependências funcionais presentes na tabela R, como descrita anteriormente.

(1) $X \rightarrow Y Z W$

(2) $X \rightarrow W$

(3) $X W \rightarrow Y W$

(4) $X Y Z W \rightarrow X Y$

(5) $Y \rightarrow Z$

À luz dos axiomas da teoria de projeto de bancos de dados aplicáveis nesse caso, é correto concluir que, dentre essas dependências inferidas:

- a) somente 2 é válida;
- b) somente 1, 2 e 5 são válidas;
- c) somente 1, 2, 3 e 4 são válidas;
- d) somente 2, 3, 4 e 5 são válidas;
- e) todas são válidas.

23. (FGV/AUFC (TCU)/TCU/Controle Externo/Auditoria Governamental/2022) Para a questão a seguir, considere uma tabela relacional R, com atributos W, X, Y, Z, e o conjunto de dependências funcionais identificadas para esses atributos.

$X \rightarrow Y$

$X \rightarrow Z$

$Z \rightarrow X$

$Z \rightarrow W$

Com referência à tabela R, definida anteriormente, considere o esboço de um comando SQL para a criação da tabela.

```
create table R (
```

```
  X int not null,
```

```
  Y int not null,
```

```
  W int not null,
```

```
  Z int not null,
```

```
  constraint ...
```

```
)
```

De acordo com as dependências funcionais de R, e com a Forma Normal de Boyce-Codd, a

definição correta das chaves (por meio de constraints) aplicáveis e necessárias para essa tabela deveria ser:

a)

constraint c1 unique (X)

constraint c2 unique (Z)

b)

constraint c1 unique (X, Z)

c)

constraint c1 unique (X, Y, W)

d)

constraint c1 unique (X, Y, Z)

e)

constraint c1 unique (X)

constraint c2 unique (Y)

constraint c3 unique (Z)

24. (FGV/Prof (SEAD AP)/SEAD AP/Educação Básica Profissional/Informática/2022) No projeto de bancos de dados relacionais, a identificação das dependências funcionais é fundamental no processo de

a) dimensionamento físico do banco de dados.

b) estabelecimento dos privilégios de acesso.

c) identificação das chaves estrangeiras.

d) normalização das tabelas.

e) otimização das consultas.

25. (FGV/Ana MP (MPE GO)/MPE GO/Analista em Informática/2022) No Campeonato Brasileiro de Futebol, todos os times jogam entre si duas vezes, alternando os papéis de mandante e visitante. Um time não pode jogar mais de uma vez na mesma data.

Nesse contexto, considere uma tabela de bancos de dados que armazene simplificada essas informações, com as seguintes colunas:

- time_mandante

- time_visitante
- data
- gols_mandante
- gols_visitante

À luz dessas premissas, assinale a lista com todas as combinações de colunas que devem, necessária e exclusivamente, ser estabelecidas como restrições (primary key ou unique) para atender à Forma Normal Boyce-Codd.

- a) (time_mandante, time_visitante)
- b) (time_mandante, data)
- (time_visitante, data)
- c) (time_mandante, data, gols_mandante)
- (time_visitante, data, gols_visitante)
- d) (time_mandante, time_visitante)
- (time_visitante, time_mandante)
- e) (time_mandante, time_visitante)
- (time_mandante, data)
- (time_visitante, data)

26. (FGV/Ag TE (SEFAZ BA)/SEFAZ BA/Administração e Finanças/2022) Leia o fragmento a seguir.

“Uma tabela está na _____ se, e somente se, para cada _____, onde X e A são atributos simples ou compostos, uma das duas condições precisam ser mantidas: ou o atributo X é uma _____, ou o atributo A é uma chave candidata. Se o atributo A é membro de uma chave candidata”.

Assinale a opção cujos itens completam corretamente as lacunas do fragmento acima.

- a) forma normal boyce-codd – dependência multivalorada – chave primária.
- b) forma normal boyce-codd – dependência funcional não trivial $X \rightarrow A$ – chave primária.
- c) terceira forma normal – dependência funcional trivial $X \rightarrow A$ – chave candidata.
- d) terceira forma normal – dependência funcional não trivial $X \rightarrow A$ – superchave.
- e) quarta forma normal – dependência funcional trivial $X \rightarrow A$ – chave candidata.

27. (FGV/ES (SEMSA Manaus)/Pref Manaus/Analista de Banco de Dados/2022) Suponha a existência dos atributos V, W, X, Y, Z numa tabela de banco de dados relacional, e considere as seguintes assertivas sobre dependências funcionais e seus axiomas.

I. Se $W \rightarrow X$ e $W \rightarrow Y$ então $X \rightarrow Y$

II. Se $X \rightarrow Y$ então $XW \rightarrow YW$

III. $XY \rightarrow X$

IV. Se $X \rightarrow Y$ e $Y \rightarrow Z$ e $Z \rightarrow V$ então $X \rightarrow V$

Estão corretas as afirmativas

- a) I, II, III e IV.
- b) I e III, somente.
- c) I, II e IV, somente.
- d) II e IV, somente.
- e) II, III e IV, somente.

28. (FGV/AS (SEMSA Manaus)/Pref Manaus/Programador de Computador/2022) Em todos os comandos, supõe-se que NULL é tratado como um valor desconhecido (unkown).

T1		T2		T3		
A	B	C	D	A	C	E
1	3	100	42	1	100	12
2	2	110	55	1	110	12
4	2	120	33	2	100	NULL

Tomando por base apenas as instâncias das tabelas do banco de dados EXEMPLO, anteriormente especificado, é possível confirmar a existência da dependência funcional

- a) $A \rightarrow B$
- b) $A \rightarrow C$
- c) $B \rightarrow A$
- d) $C \rightarrow A$

e) $E \rightarrow C$

29. (FGV/Ana (FunSaúde CE)/FunSaúde CE/ TI e Infraestrutura de Informática/2021) Considere a definição e as instâncias das tabelas de bancos de dados CLUBE e JOGO exibidas a seguir.

nome
Barcelona
Boca Juniors
The Strongest

JOGO

mandante	visitante	golsM	golsV
Barcelona	Boca Juniors	1	0
Barcelona	The Strongest	NULL	NULL
Boca Juniors	Barcelona	0	0
Boca Juniors	The Strongest	3	0
The Strongest	Barcelona	2	0
The Strongest	Boca Juniors	2	0

Cada clube deve jogar quatro vezes, duas como mandante e duas como visitante.

As colunas golsM e golsV registram o número de gols dos times mandantes e visitantes, respectivamente, em cada jogo. Ambas são nulas enquanto o jogo não for realizado.

Dependências funcionais têm um papel importante na teoria de bancos de dados. Tomando por base a instância da tabela JOGO, definida anteriormente, assinale a dependência funcional que pode ser corretamente depreendida.

a) mandate $\rightarrow$ golsM

b) mandate, visitante $\rightarrow$ golsM

c) golsM $\rightarrow$ mandante

d) golsM, golsV $\rightarrow$ mandante, visitante

e) visitante $\rightarrow$ gols, golsV

30. (FGV/Ana TI (BANESTES)/BANESTES/Desenvolvimento de Sistemas/2021) Considere um banco de dados que contém uma tabela com os atributos CPF, Nome, RG e Endereço, com milhões de registros. Sabe-se que cada pessoa tem seus próprios CPF e RG e um único nome, embora possa haver homônimos.

Sendo assim, a dependência funcional que NÃO existe entre os atributos dessa tabela é:

a) CPF $\rightarrow$ Nome

b) CPF $\rightarrow$ RG

c) Endereço $\rightarrow$ CPF

d) RG $\rightarrow$ Endereço

e) RG $\rightarrow$ Nome

31. (FGV/ATCE (TCE,AM)/TCE AM/Auditoria de Tecnologia da Informação/2021) Na questão, considere uma tabela de banco de dados T cuja instância é exibida a seguir.

A	B	C	D
12	2	3	1
14	3	8	2
18	2	9	3
21	5	4	4

Supondo-se que a tabela T (descrita anteriormente) esteja normalizada até a forma normal Boyce-Codd, a dependência funcional que NÃO se verifica nessa tabela é:

a) A $\rightarrow$ B

b) B $\rightarrow$ D

c) C $\rightarrow$ B

d) C $\rightarrow$ D

e) A, D $\rightarrow$ B

GABARITO

1. B	12. C	23. A
2. D	13. C	24. D
3. C	14. E	25. E
4. B	15. C	26. D
5. C	16. C	27. E
6. C	17. D	28. A
7. A	18. B	29. B
8. C	19. A	30. C
9. B	20. E	31. B
10. E	21. A	
11. B	22. C	

ESSA LEI TODO MUNDO CONHECE: PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.



1 Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.



2 Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).



3 Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).



4 Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).



5 Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).



6 Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).



7 Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.



8 O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.